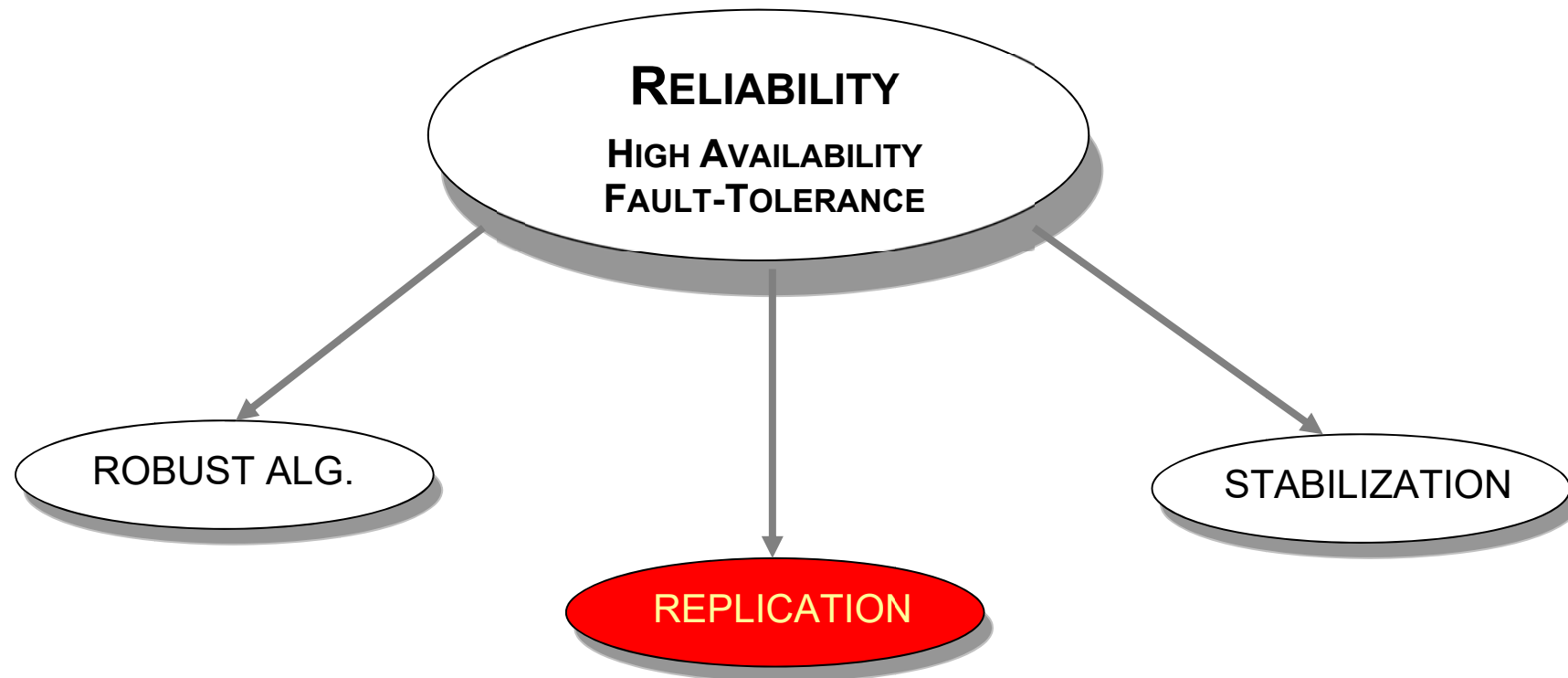
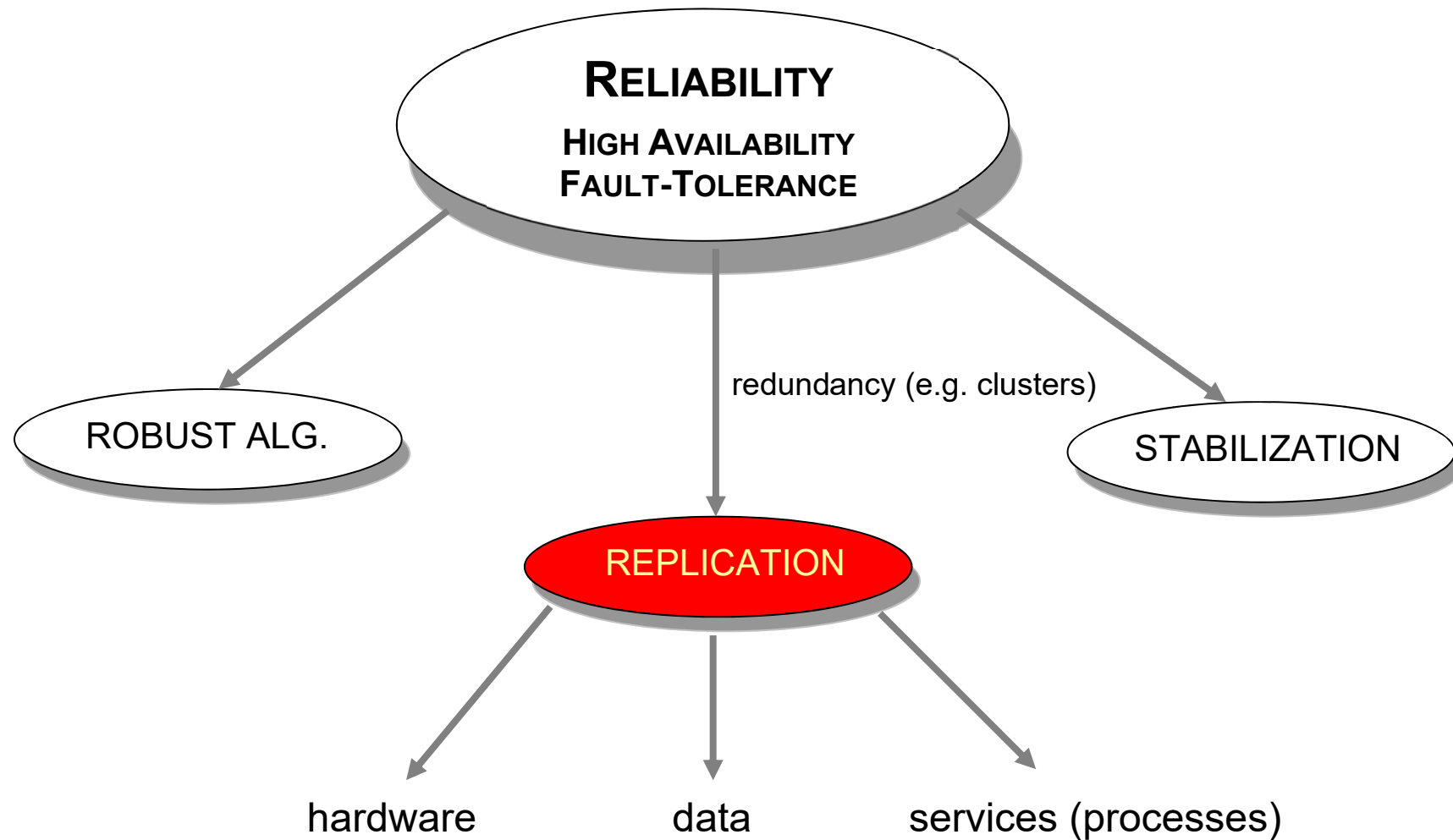


MAIN TOPICS OF THE COURSE



RELIABILITY BY REPLICATION





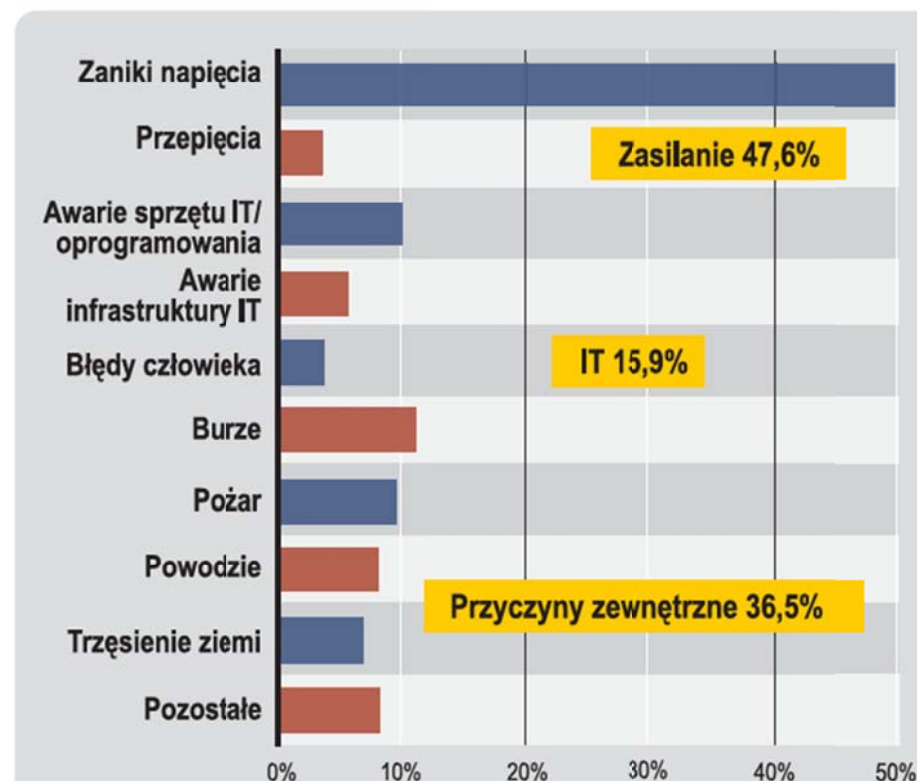
ZASILANIE

CIĄGŁOŚĆ ZASILANIA

Źródła rezerwowego zasilania

- zasilacze awaryjne UPS
- agregaty prądotwórcze
- ogniwa paliwowe

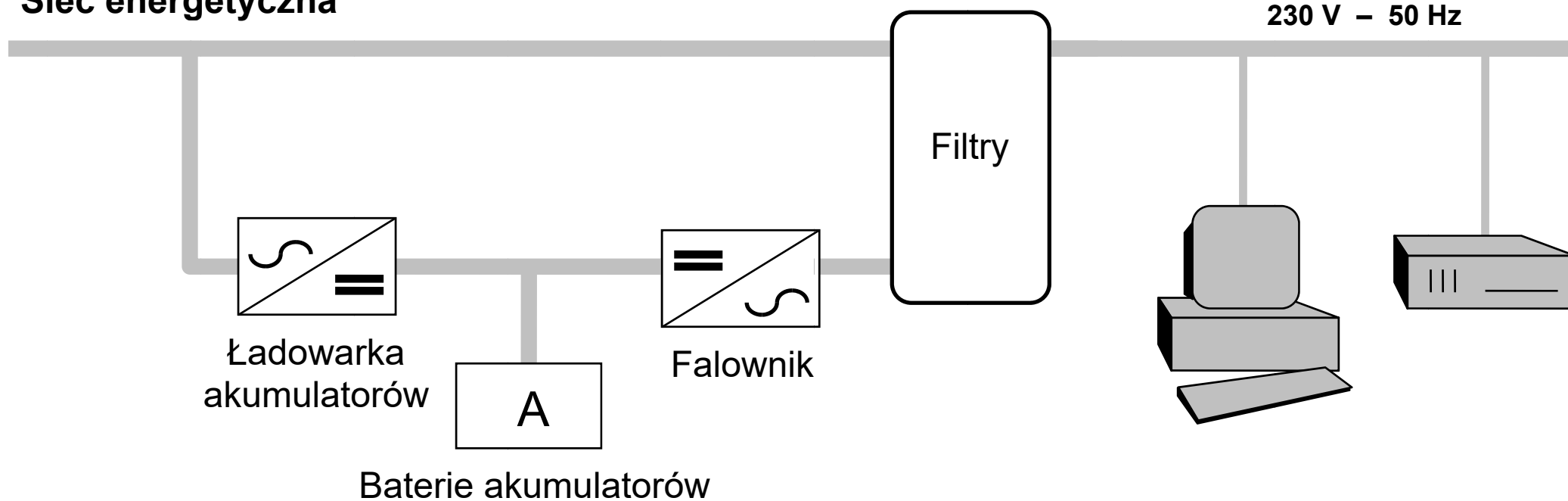
GŁÓWNE PRZYCZYNY UTRATY DANYCH W SYSTEMACH IT



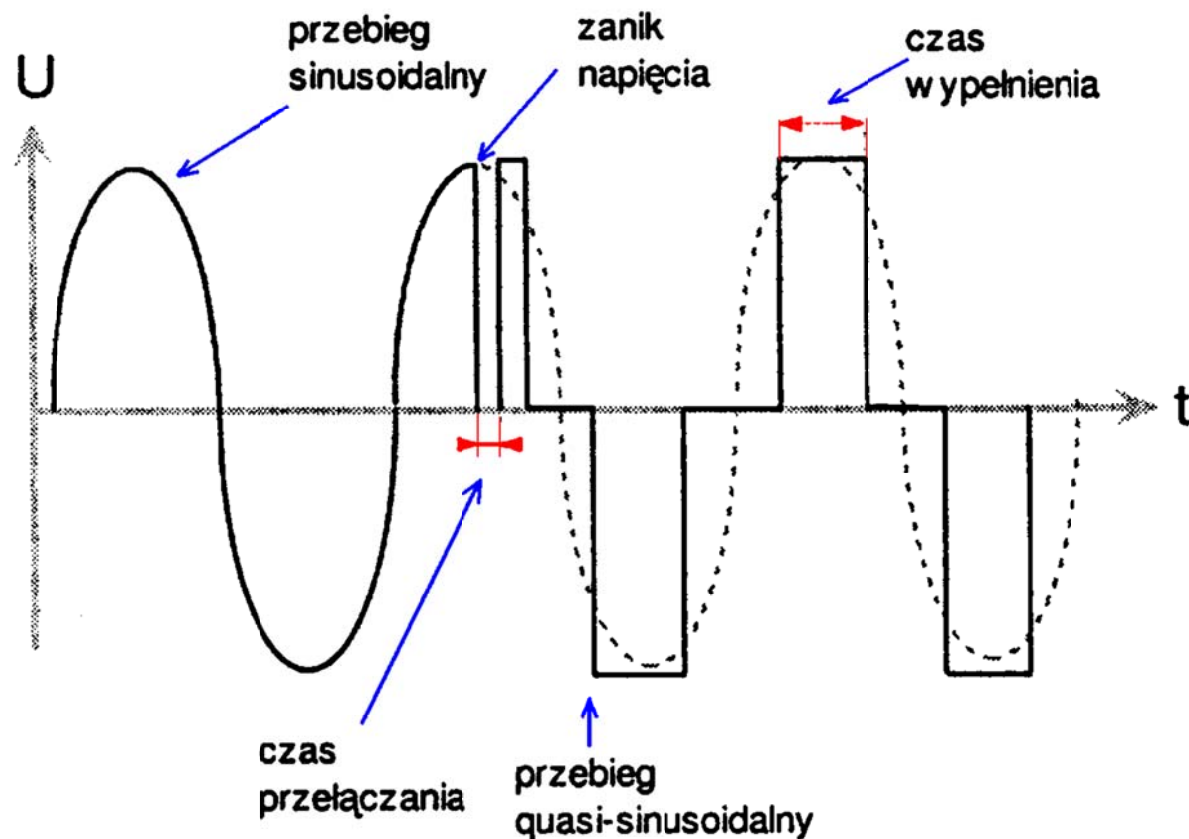
Źródło: Unterbrechungsfreie Stromversorgung European Guide/CEMEP 2nd Edition 2004

off-line UPS

Sieć energetyczna

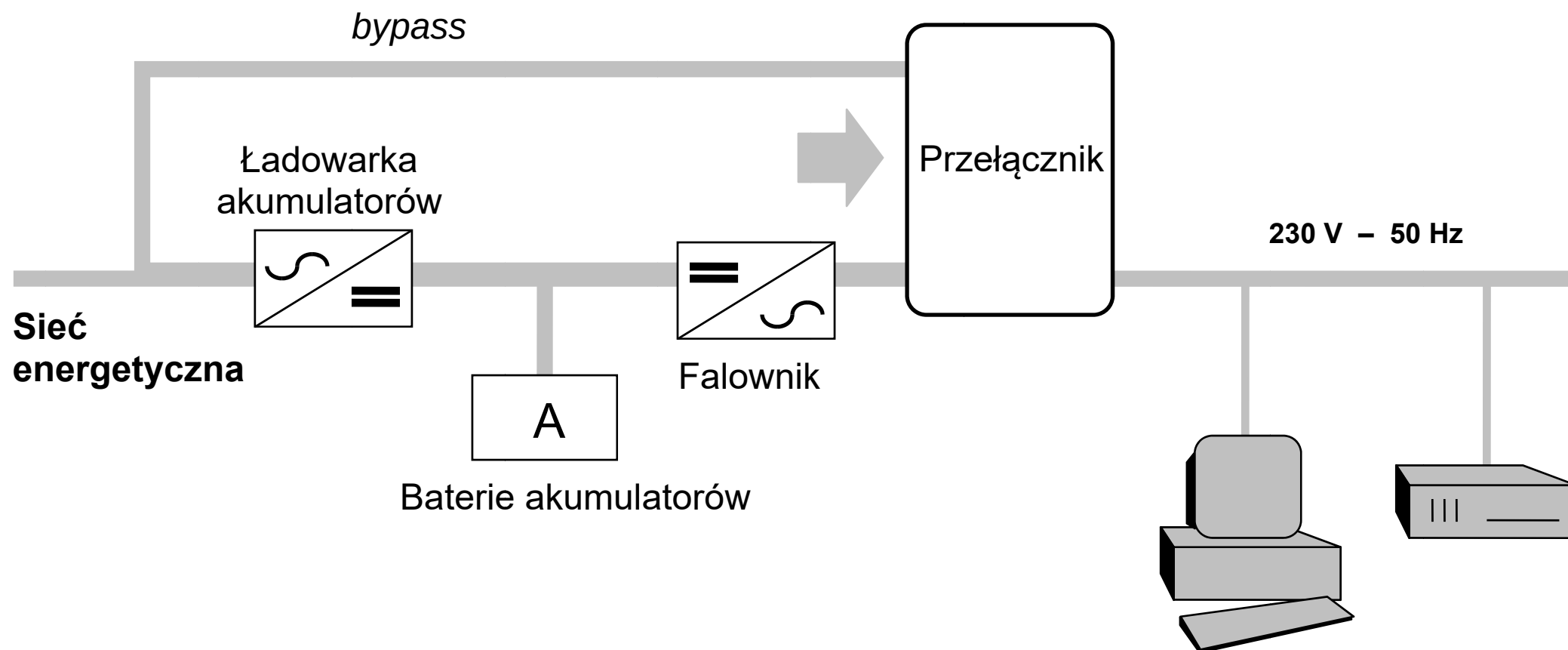


off-line UPS



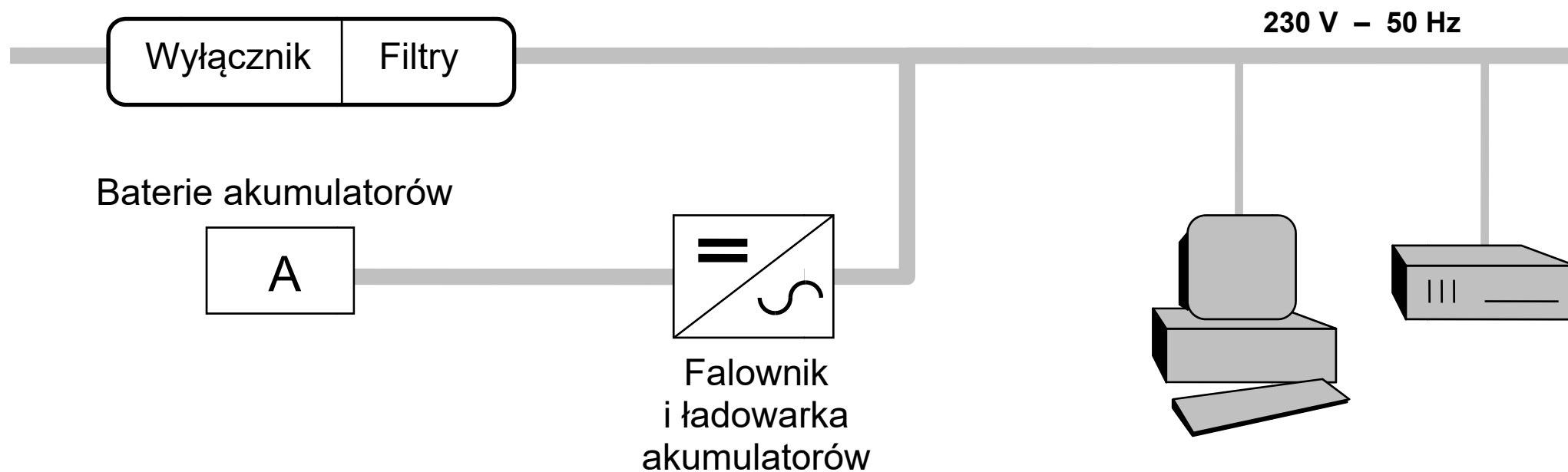
Kształt napięcia na wyjściu zasilacza awaryjnego *off-line*

on-line UPS

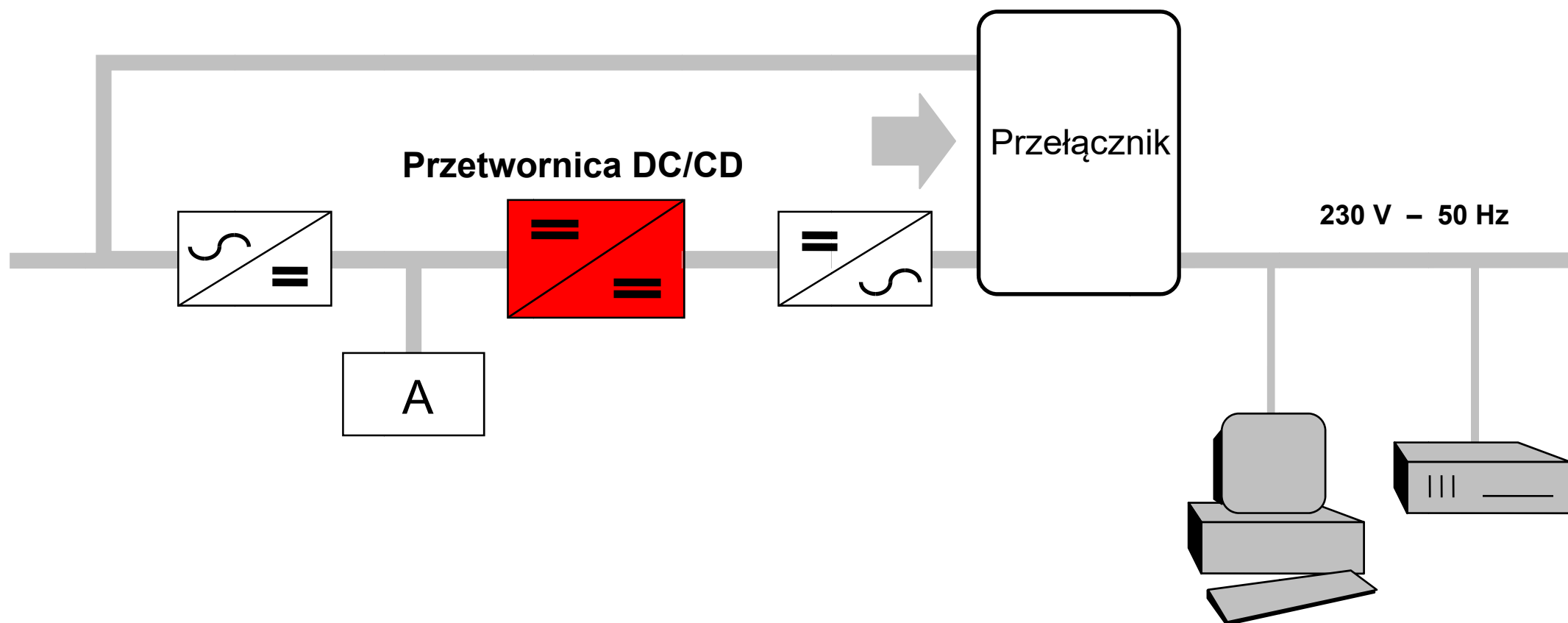


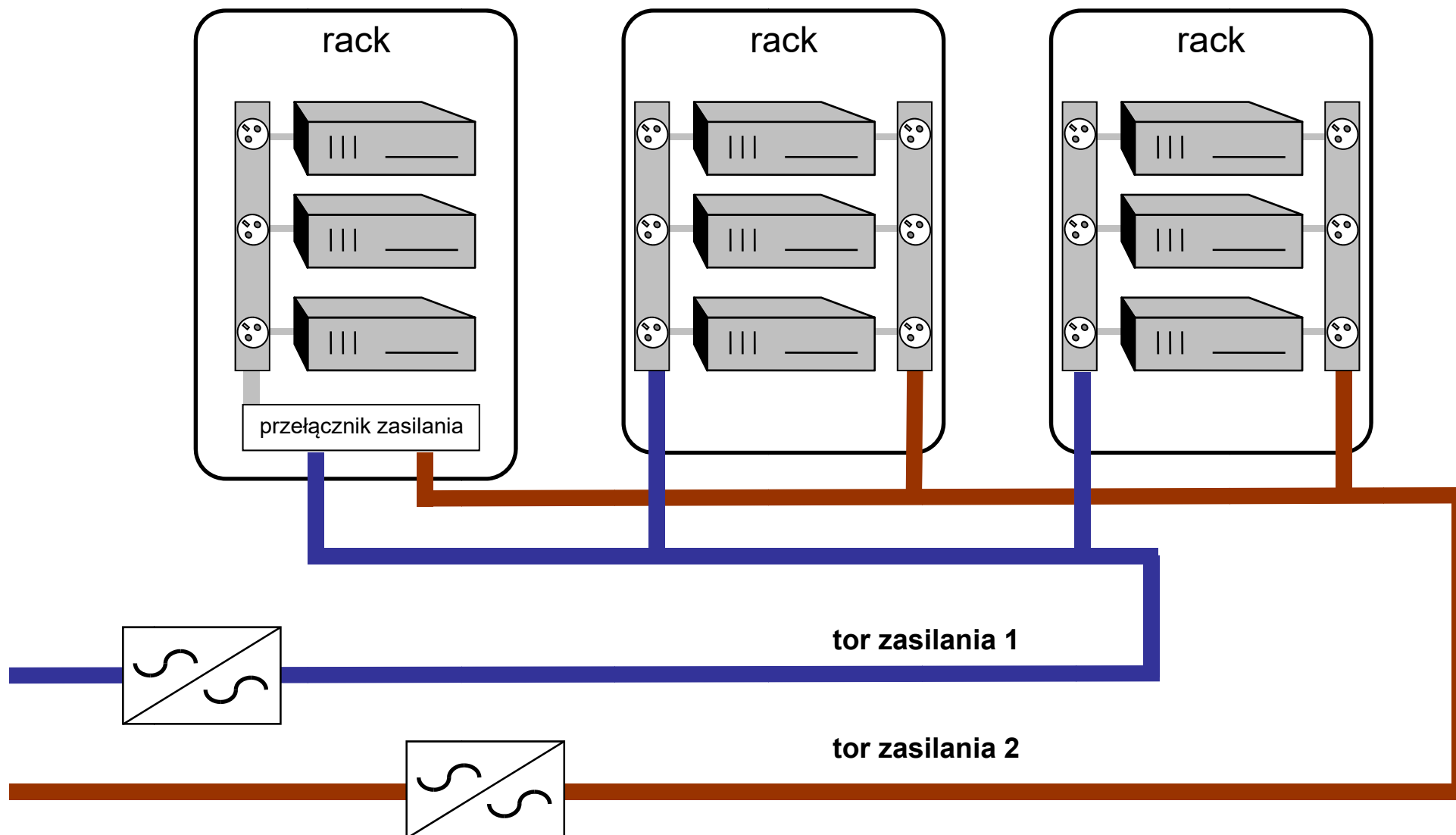
line-interactive / in-line UPS

Sieć energetyczna

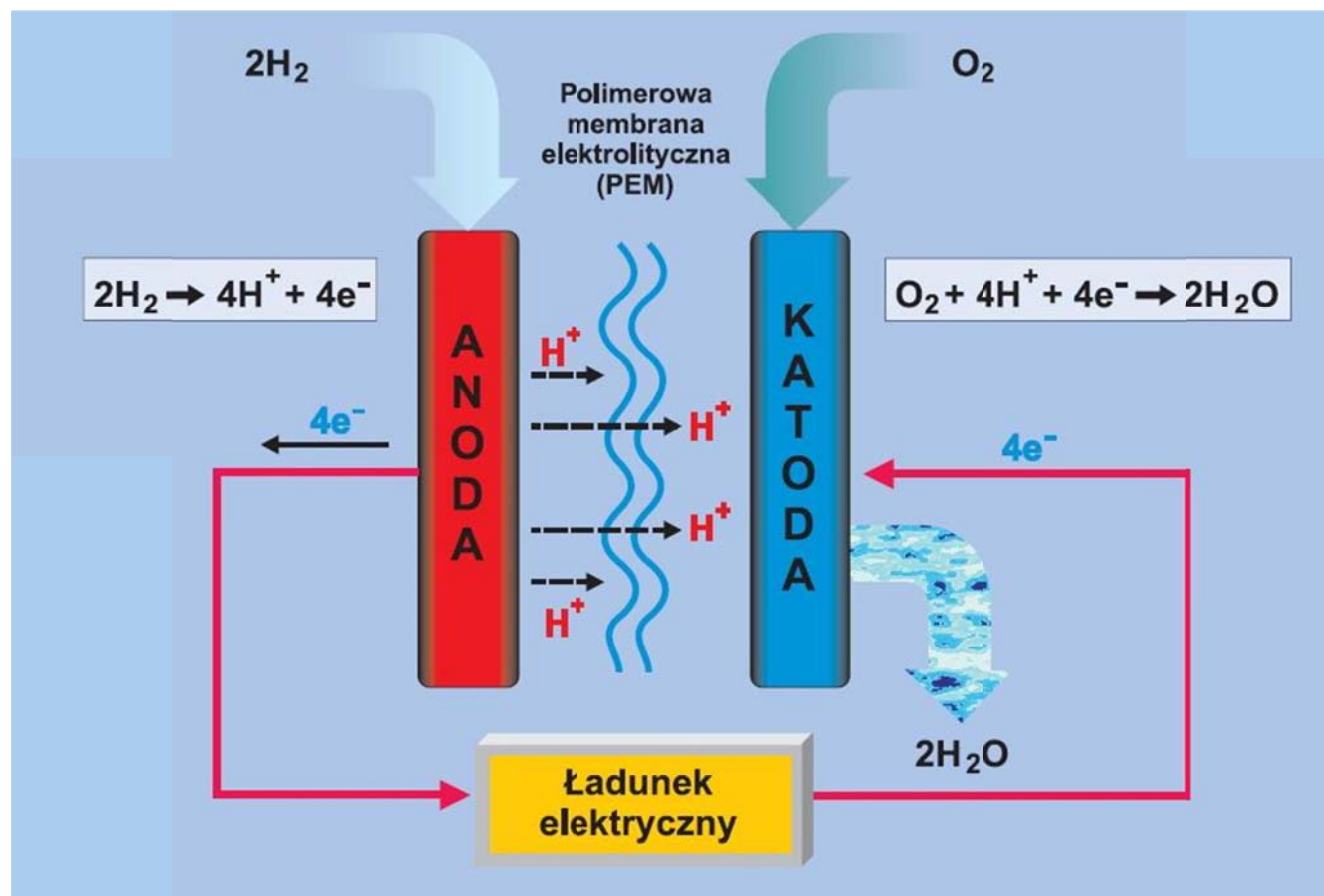


true on-line UPS





Ogniwo paliwowe



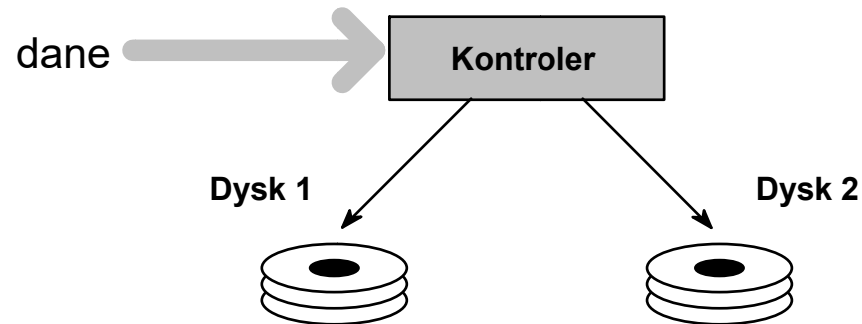
w wyniku elektrochemicznej reakcji łączenia wodoru z tlenem powstaje woda i prąd

HARDWARE

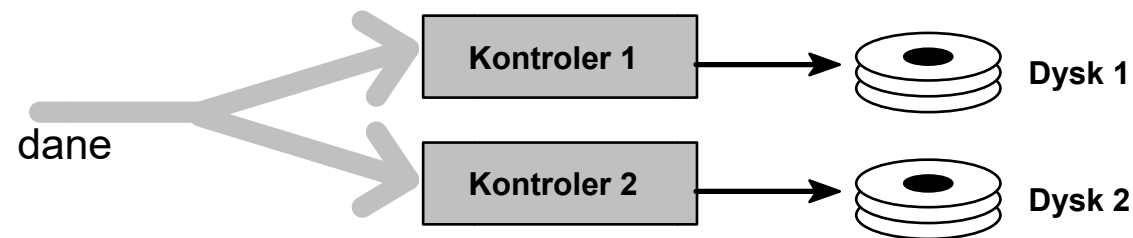
RAID (*REDUNDANT ARRAY OF INDEPENDENT DISKS*)

RAID 0 = stripping (*brak redundancji*)

RAID 1 = mirroring :

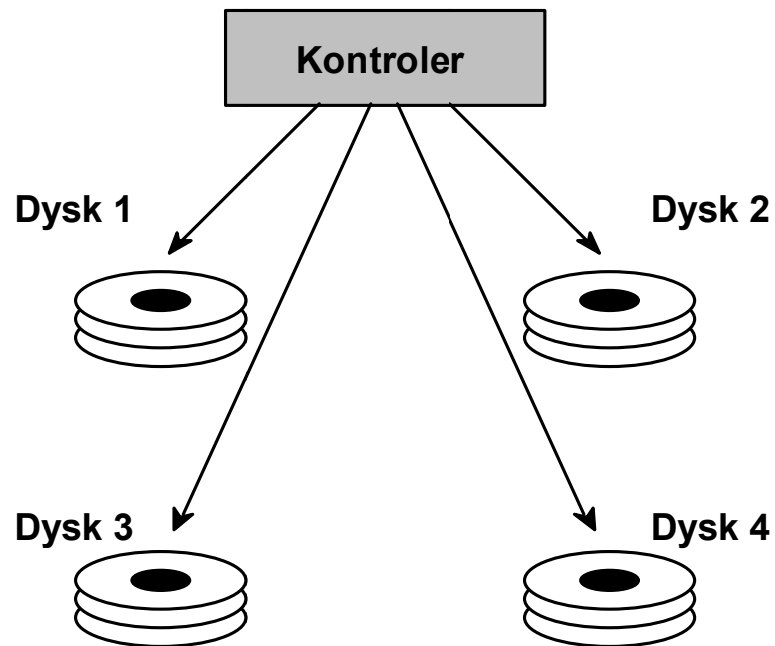


duplexing :

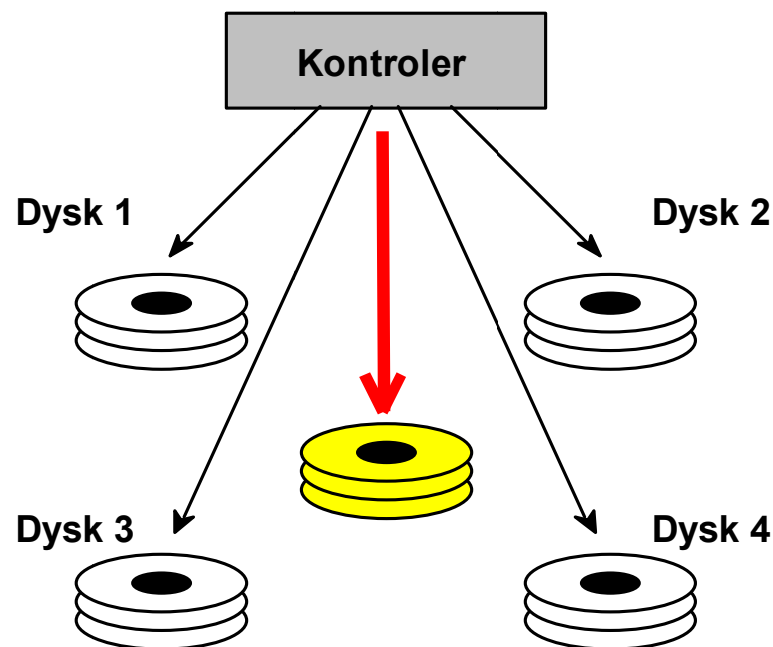


RAID 2 = stripping :

- równomierny rozdział bloku (na poziomie bitów) na wszystkie dyski
- opcjonalnie zapis sum kontrolnych ECC (kod Hamminga) w wydzielonym obszarze

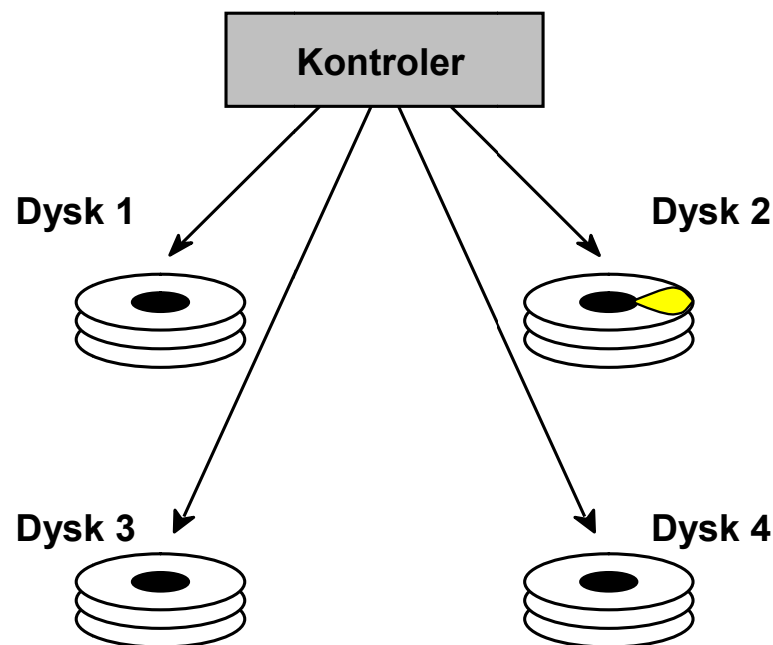


RAID 3 = stripping (bajtów) + wydzielony dysk parzystości (zawierający ECC)



RAID 4 = RAID 3 (stripping sektorów) bez synchronizacji zapisu

RAID 5 = stripping z rozproszoną „parzystością”

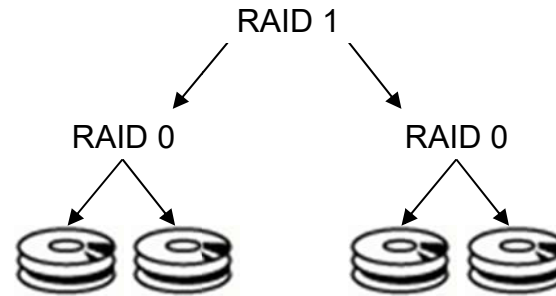


RAID 6 = kodowanie Reeda-Solomona

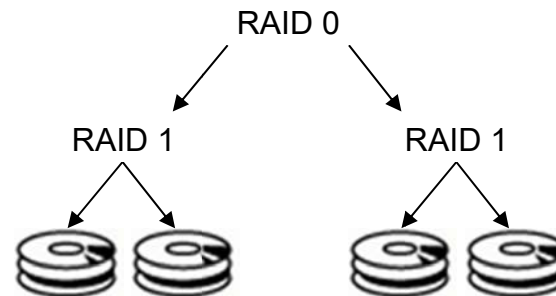
- zduplikowane kody korekcyjne – każdy blok parzystości zapisywany w 2 kopiach na 2 oddzielnych dyskach (w sposób rozproszony)
- odporność na awarie 2 dysków jednocześnie

RAID	Minimalna liczba dysków	Dostępna przestrzeń	Liczba dysków, które mogą ulec awarii bez utraty danych
RAID 0	2	N	0
RAID 1	2	1	$N-1$
RAID 2	3	$N - \log N$	1
RAID 3	3	$N-1$	1
RAID 4	3	$N-1$	1
RAID 5	3	$N-1$	1
RAID 6	4	$N-2$	2

RAID 0+1



RAID 10



Hot Spare

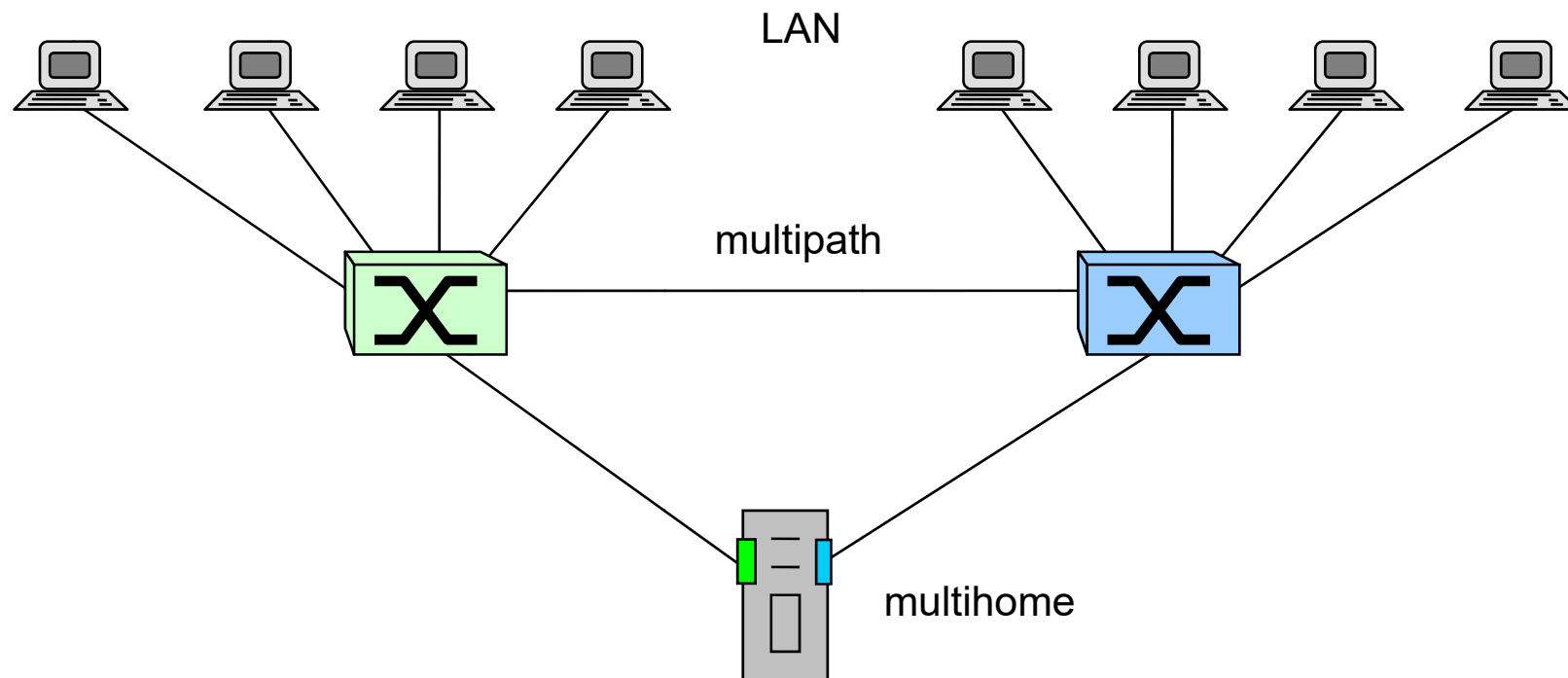
- zapasowy dysk, gotowy do użycia na wypadek awarii



KOMUNIKACJA

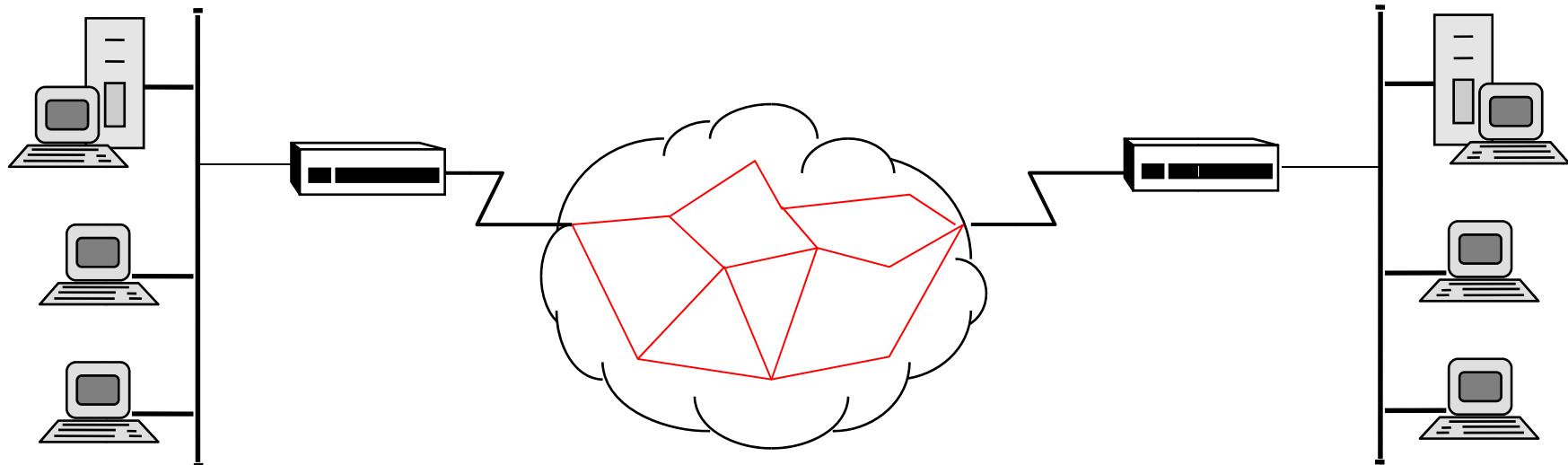
REDUNDANCJA INTERFEJSU SIECIOWEGO

Multihoming:



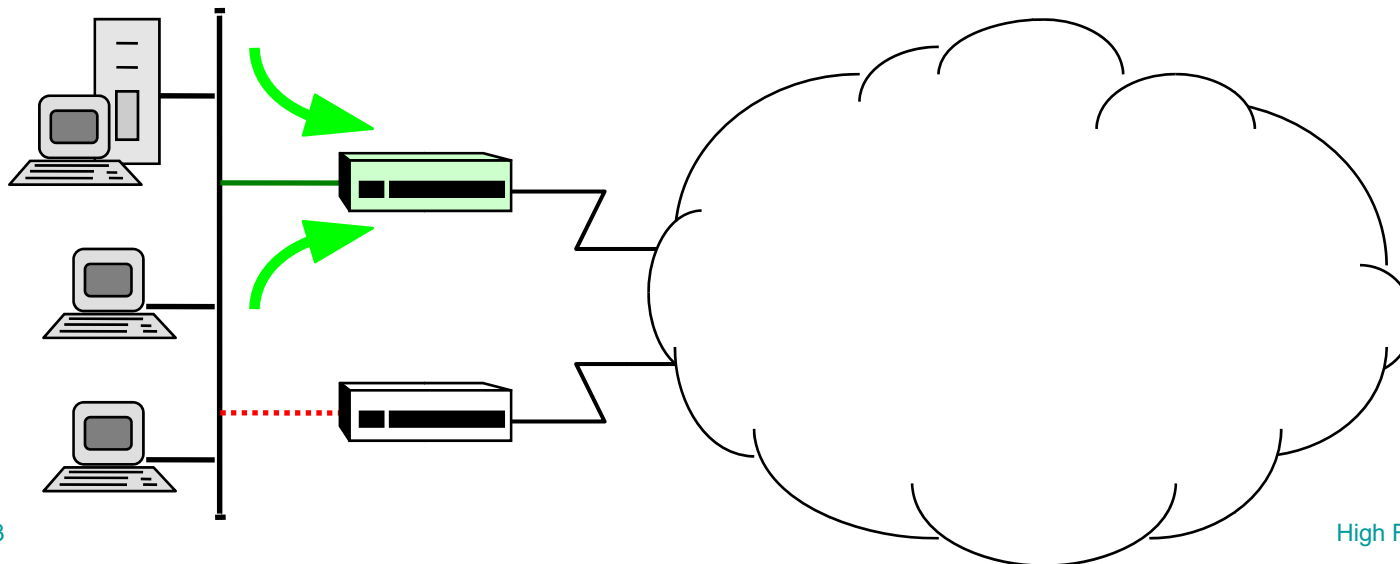
Redundancja łączy w sieci rozległej

- sieć prywatna, łącza prywatne lub dzierżawione – sami dbamy o nadmiarowość – dynamiczny routing (RIP, OSPF, EIGRP, BGP, IS-IS)
- sieć publiczna – istnieje już pewna nadmiarowość łączy operatora sieci, można wykorzystać kilku operatorów



Redundancja routerów

- **RDP** (Router Discovery Protocol) – routery mają różne IP i stacje sieciowe wybierają spośród nich jeden aktywny (zaangażowany software stacji sieciowych, wymagany ICMP)
- **HSRP** (Hot Standby Routing Protocol, RFC 2281), **VRRP** (Virtual Router Redundancy Protocol, RFC 2338), **GLBP** (Gateway Load Balancing Protocol, Cisco) – routery mają ten sam IP i dynamicznie wybierają spośród siebie jeden aktywny transparentnie dla stacji sieciowych



Redundancja routerów

HSRP / VRRP / GLBP

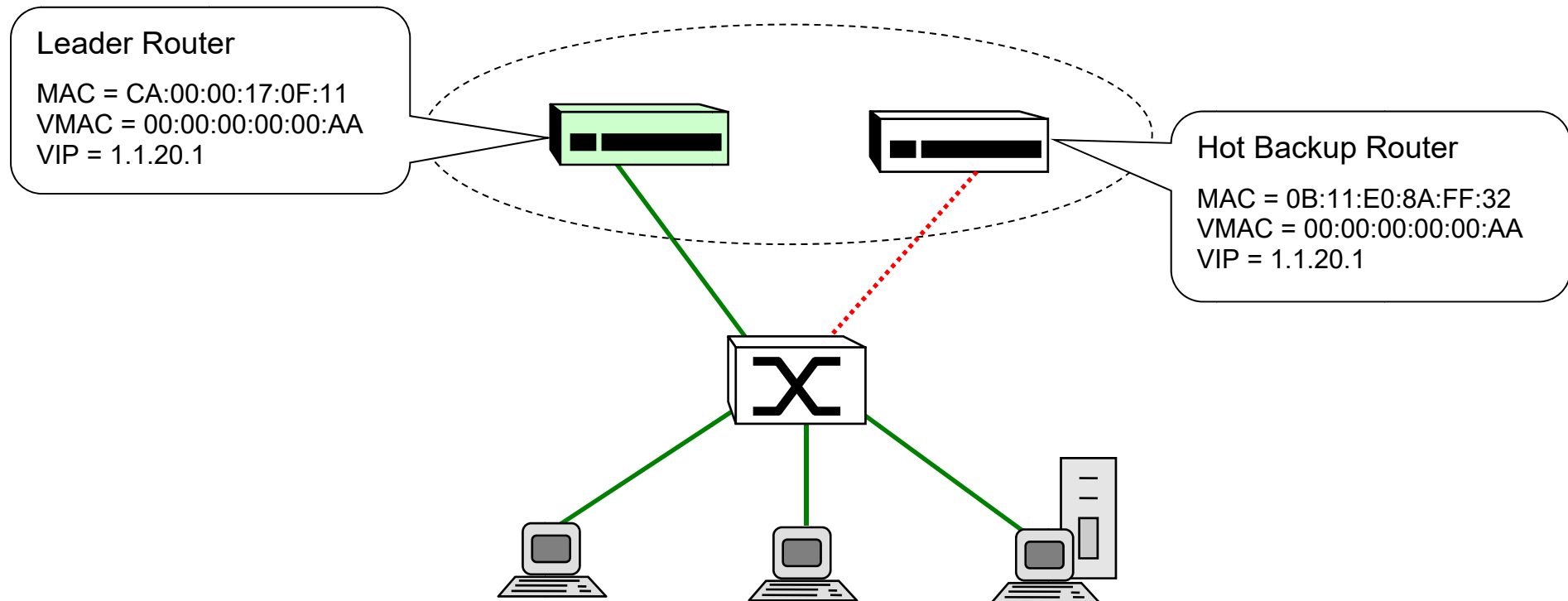
- HSR Group ("Phantom Router") – grupa routerów o wspólnym adresie VIP i VMAC
- Leader Router – aktualnie obsługujący pakiety
- Hot Backup Routers – monitorują lidera
- awaria lidera – STP (Spanning Tree Protocol) wybiera nowego

Trasy równoległe (GLBP)

- połączenia redundantne nie stanowią tylko zabezpieczenia, lecz są wykorzystywane na bieżąco (równoważenie obciążenia tras – *load balancing*)

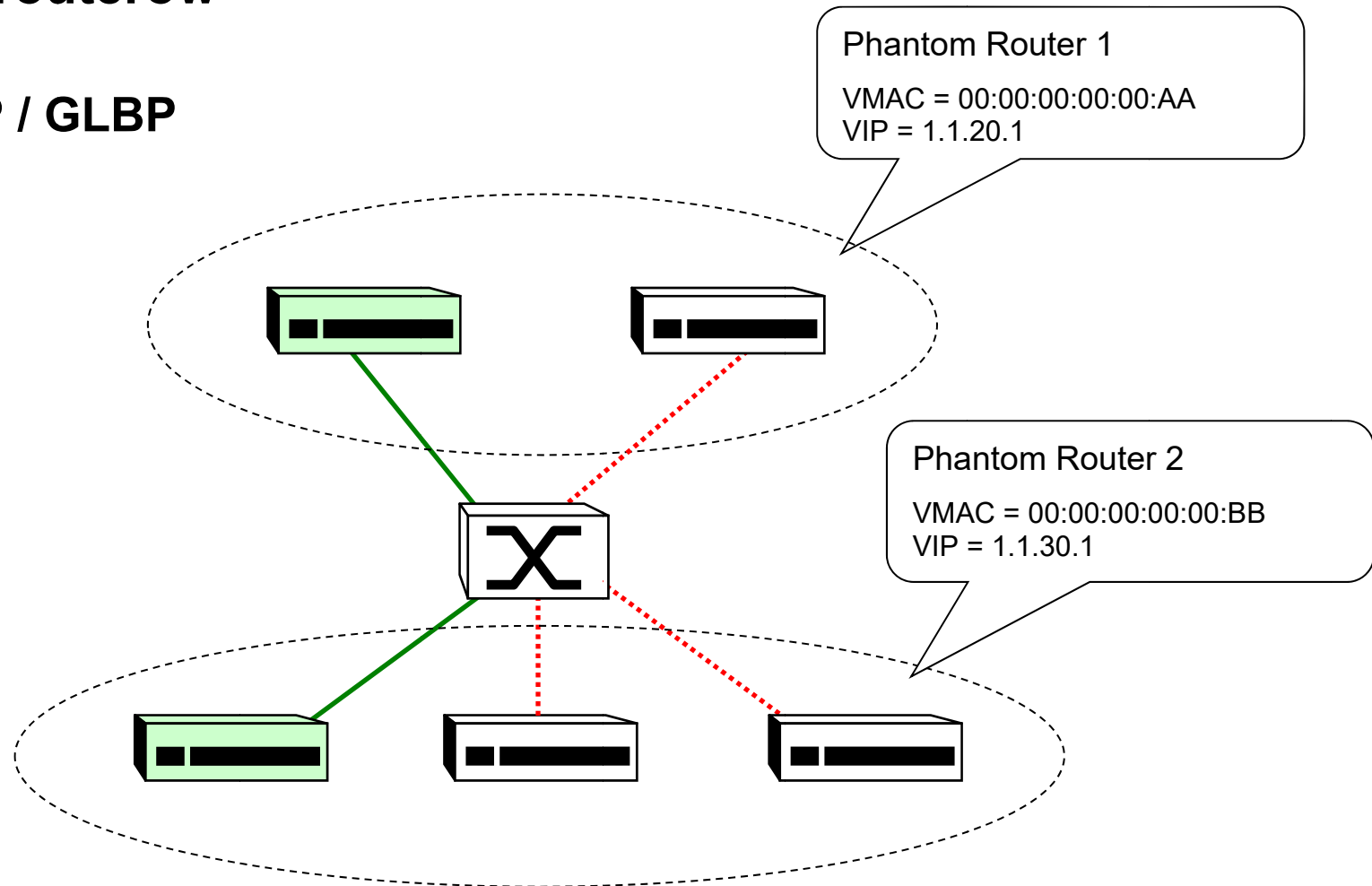
Redundancja routerów

HSRP / VRRP / GLBP



Redundancja routerów

HSRP / VRRP / GLBP





PRZETWARZANIE

REDUNDANCJA SERWERÓW

Serwery wieloprocessorowe

- architektura równoległa: wiele procesorów – jedna wspólna pamięć (globalna)
- masywna równoległość – duża liczba procesorów
- NUMA: wiele procesorów, każdy z lokalną pamięcią
- przetwarzanie aplikacji równoległych

Serwery lustrzane

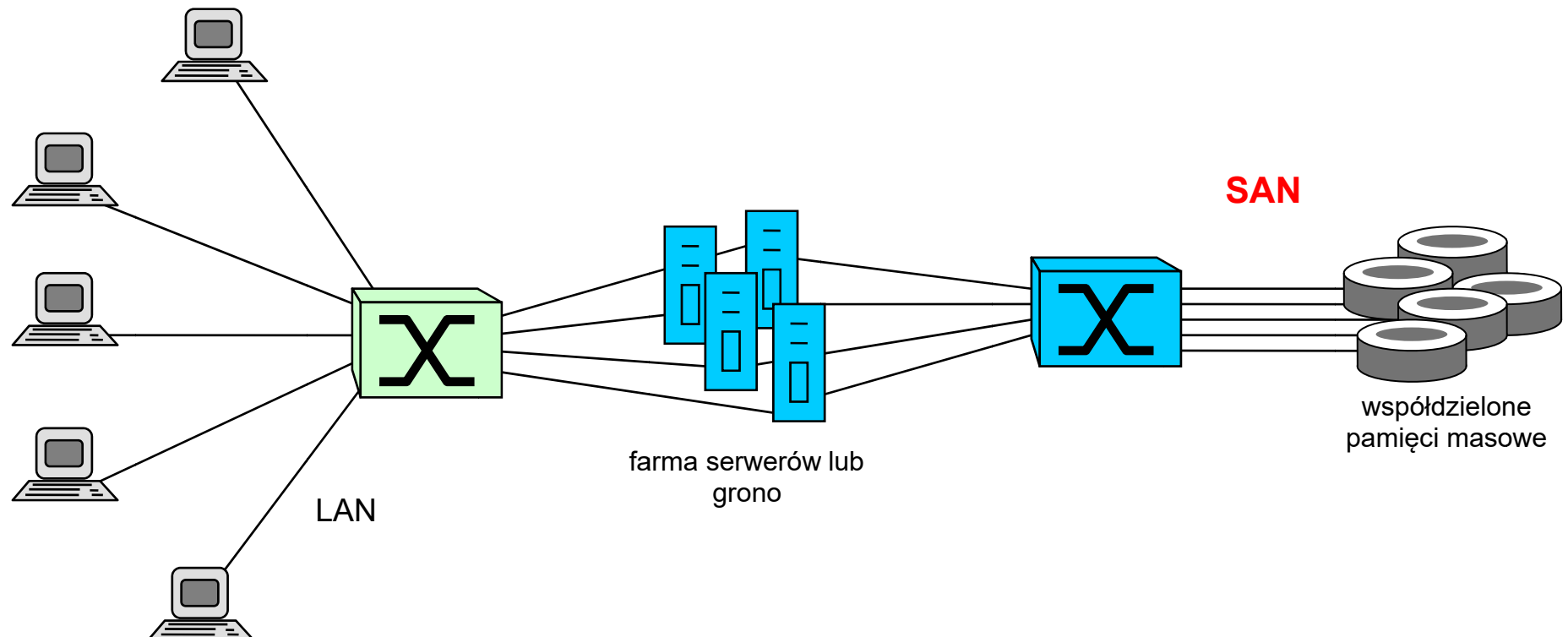
- serwer podstawowy + kopia lustrzana (*fail-over*)

Grono serwerów (*cluster*)

- po awarii dowolnego serwera jego funkcje może przejąć inny

Storage Area Network (SAN)

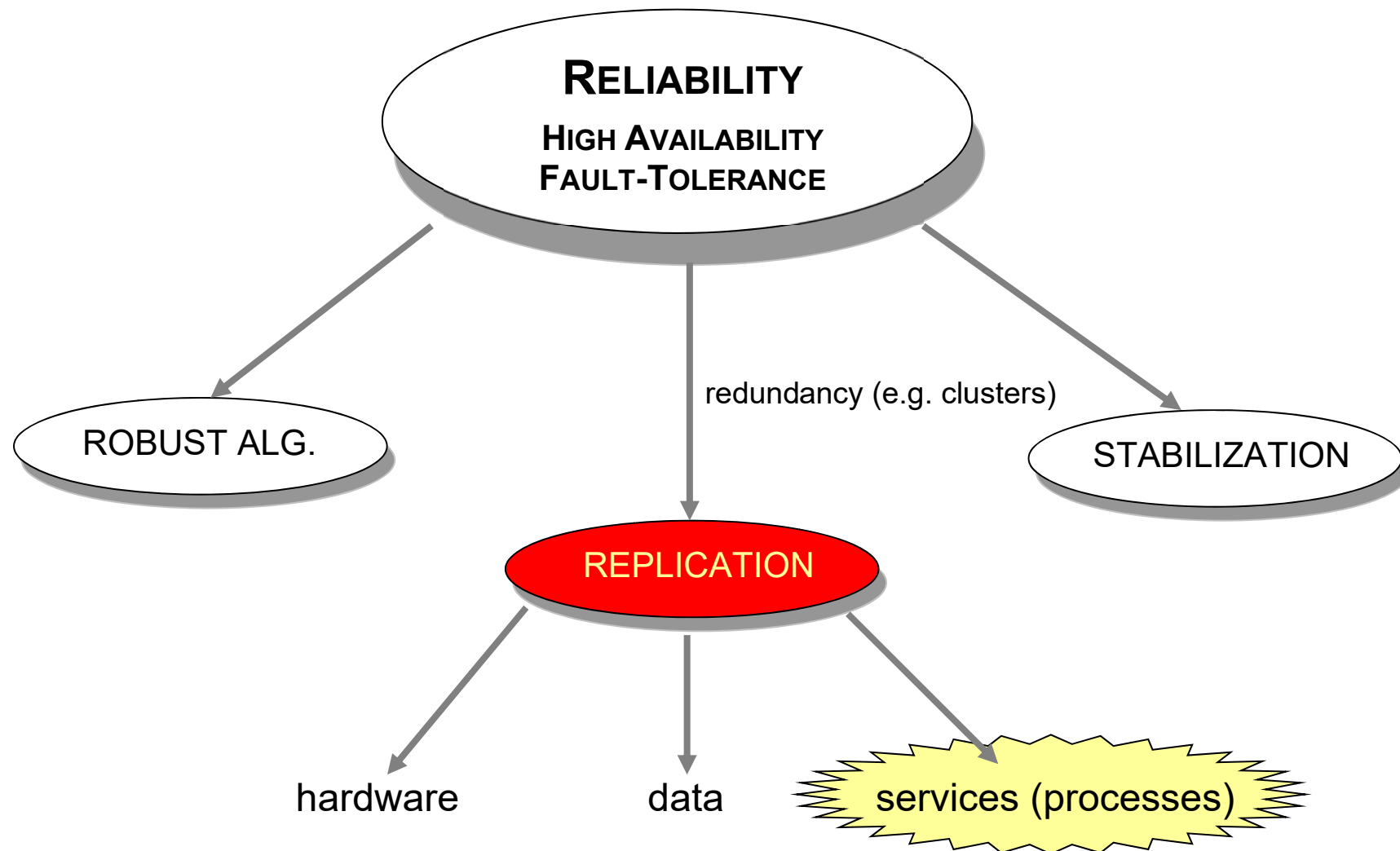
- * szybkie łącza: Serial Attached SCSI (SAS), AoE, iSCSI lub Fibre Channel
- * zasoby dyskowe są współdzielone na równych prawach przez wszystkie serwery (NFS = Network File System, CIFS = Common Internet File System, AFS-Coda)





PROCESY

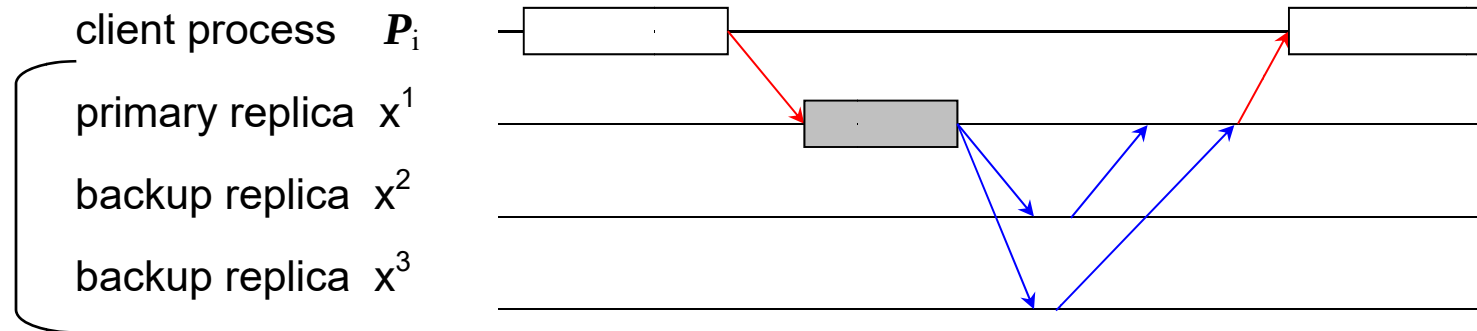
RELIABILITY BY REPLICATION



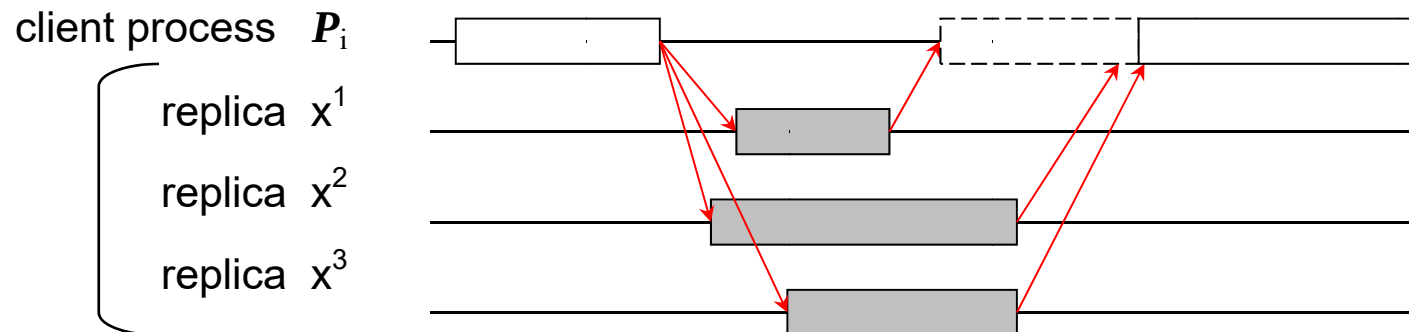
GROUP COMMUNICATION SYSTEMS: Isis, Horus, Totem, ..., OpenAIS, Corosync

Process replication techniques:

Passive replication = primary-backup:



Active replication:



Nondeterminism

Active replication

- nondeterministic processing not allowed – it leads to different replies to the same client request
- even if client chooses the first response, states of distinct replicas have evolved to inconsistency

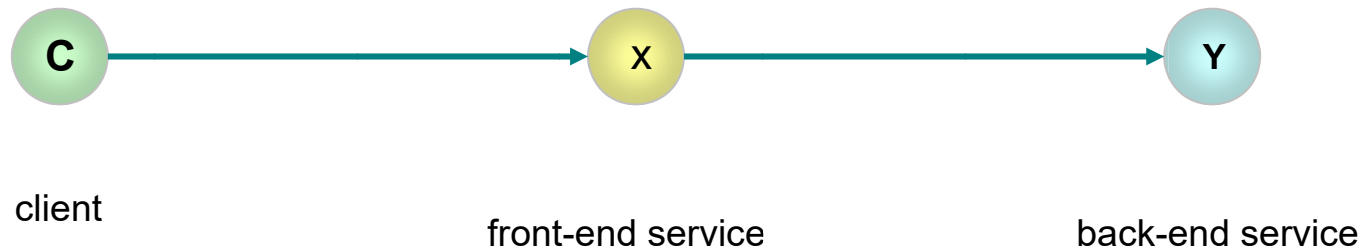
Passive replication

- nondeterministic processing is allowed since only one replica processes requests
- and updates ensure global consistency among the group members

... unless

- nested invocations are considered

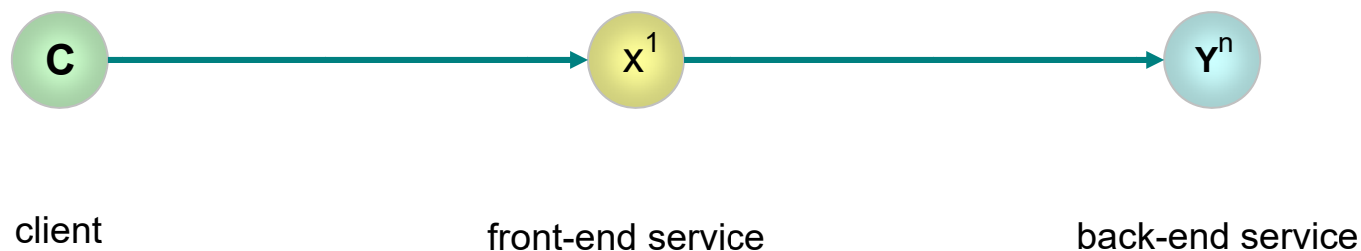
Nested invocations



Nested invocations

Passive replication

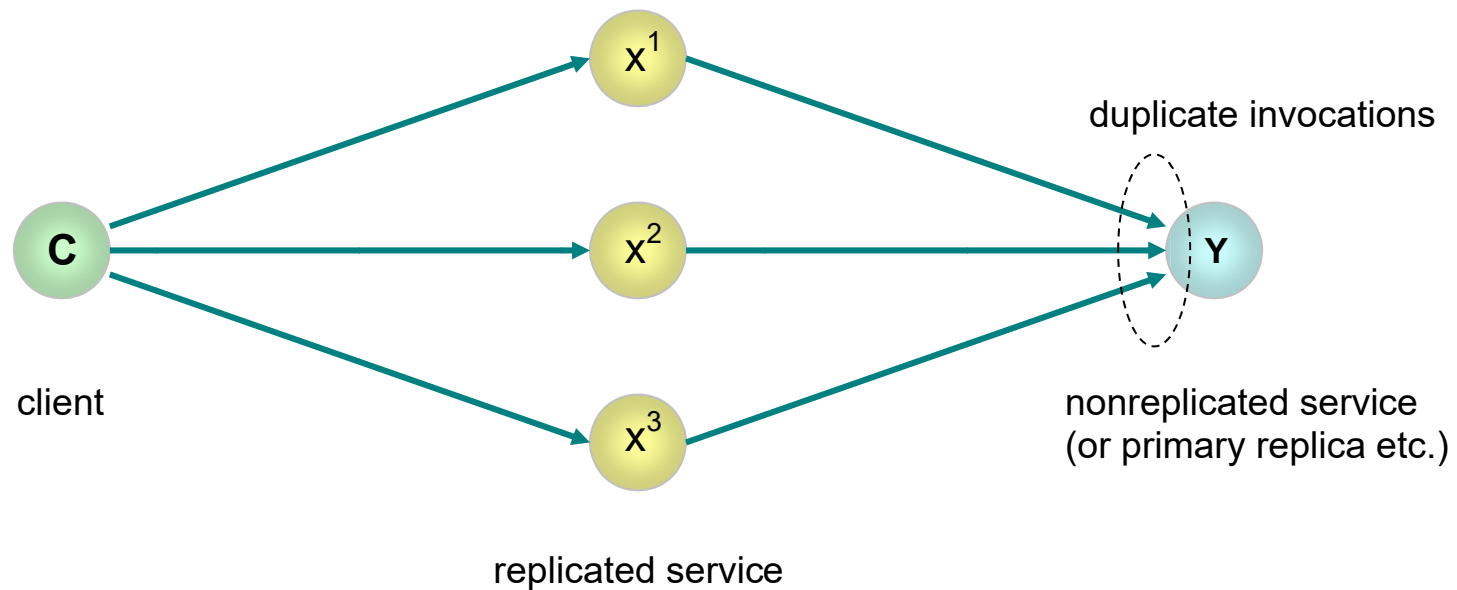
- the primary X^1 invokes another server $Y^{(n)}$
- based on a nondeterministic decision
- then, X^1 fails after receiving response from $Y^{(n)}$
- new primary X^2 invokes another server $Z^{(m)}$
- based on its own nondeterministic choice
- both $Y^{(n)}$ and $Z^{(m)}$ have been updated = inconsistent global state



Nested invocations

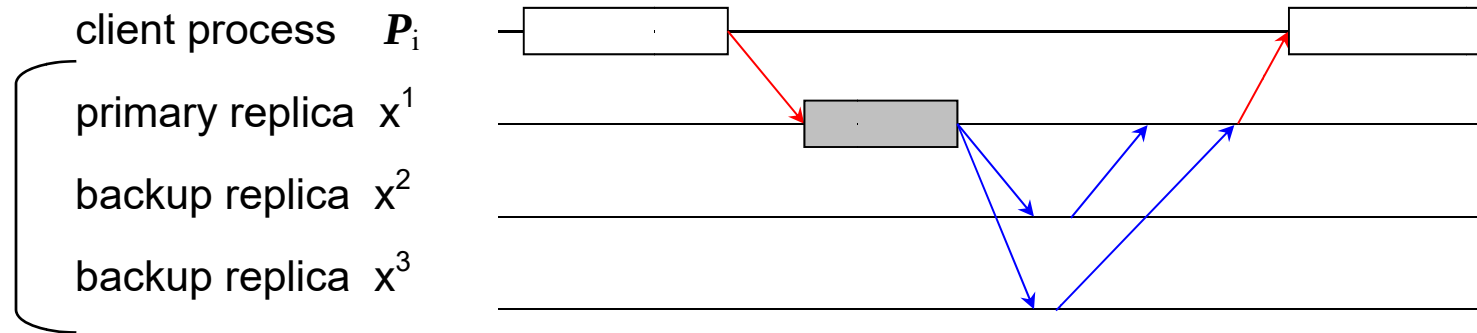
Active replication

- also active replication undergoes serious problems with nesting



Hot / warm / cold replication

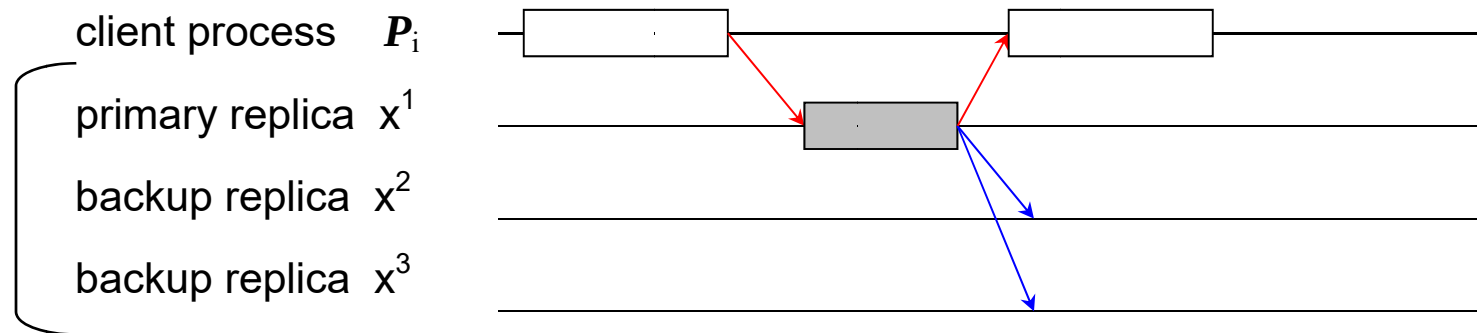
Hot-passive replication



- backups are *hot* – they always process the communication with the primary

Hot / warm / cold replication

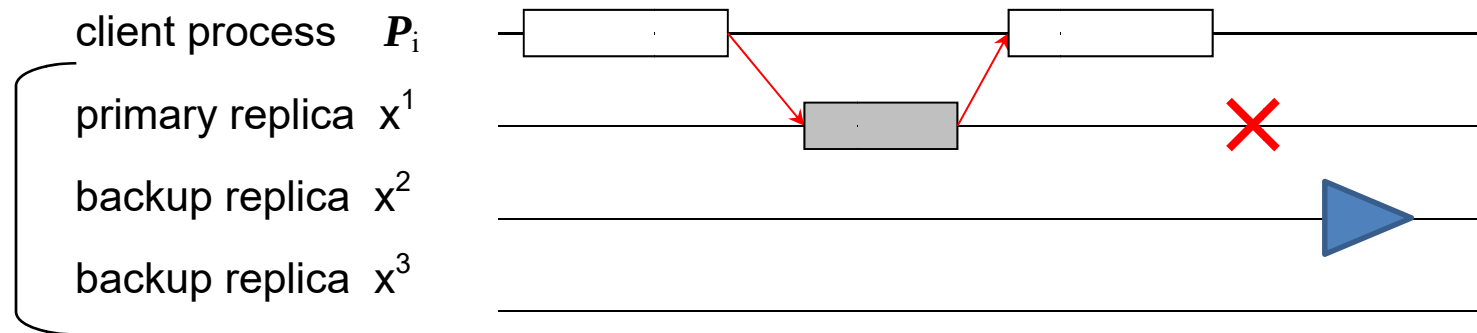
Warm-passive replication



- backup replicas are *warm* – they receive updates, but do not respond
- only eventual consistency is guaranteed after primary's failure
- essential is the choice of the new primary

Hot / warm / cold replication

Cold-passive replication



- backup replicas are cold – turned off – there is no communication with the primary
- a backup will be turned on when the primary fails
- state of the backups must be updated (periodically) to maintain consistency with the state of the primary replica
- it may require to save (i.e. *back up*) the state of the primary to enable state transfer to the backup replicas

TEORIA

i

PRAKTYKA

CLUSTERING

Different purposes of clustering [2]:

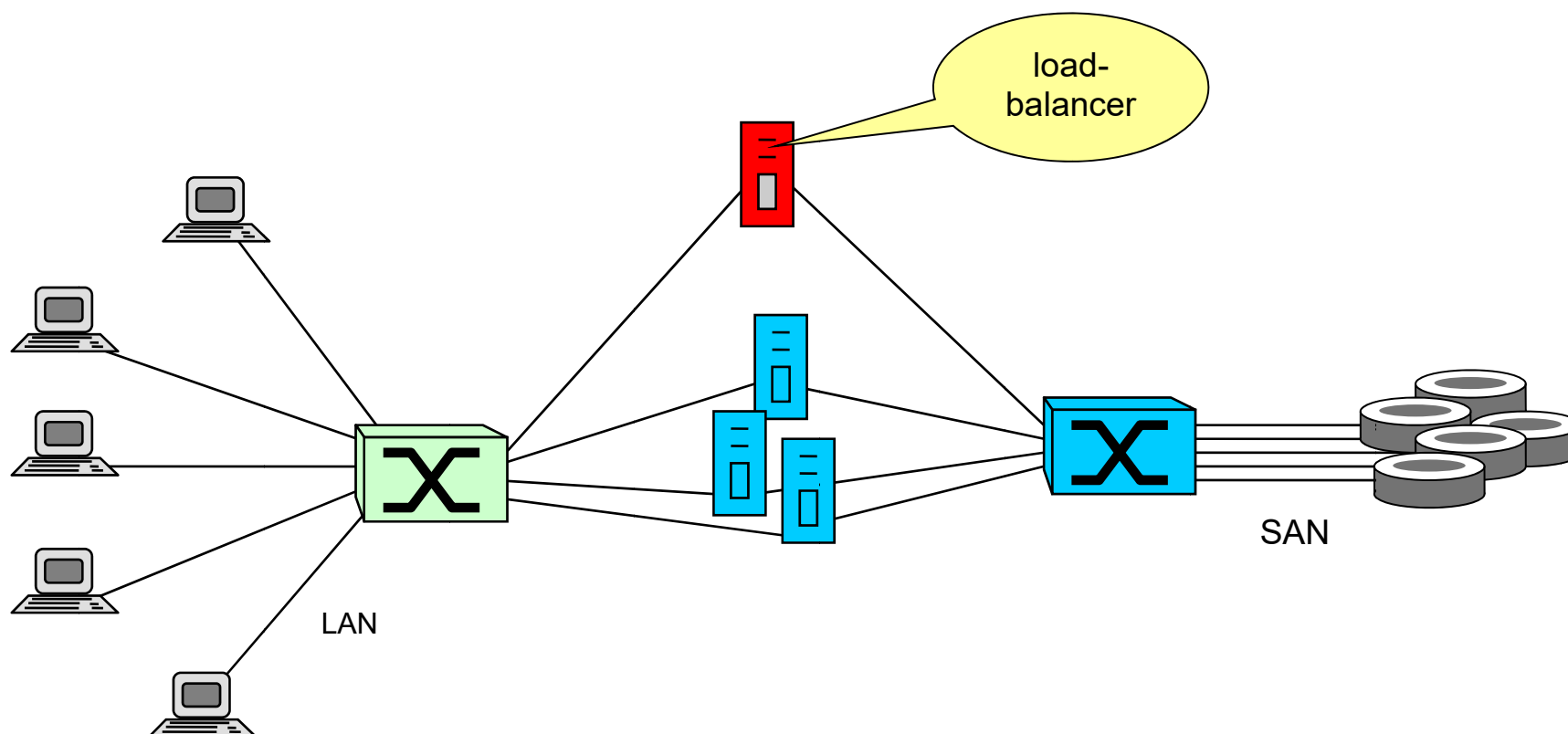
- **High performance:** Different computers work together to host one or more tasks that require lots of computing resources.
- **Load balancing:** A load balancer serves as a front end and receives requests from end users. The load balancer distributes the request to different servers.
- **High availability:** Different servers work together to make sure that the downtime of critical resources is reduced to a minimum.

High Performance Clusters:

- Used in environments that have heavy computing needs.
- E.g.: large rendering jobs or complicated scientific calculations that are too big to be handled by one single server.
- In such a situation, the work can be handled by multiple servers, to make sure it is handled smoothly and in a timely manner.
- Using that approach, multiple machines are treated by the cluster as one, and the cluster just allocates and claims the resources where they are available.
- An approach to high performance clustering in Linux is the use of a Single System Image (SSI).

Load Balancing Clusters:

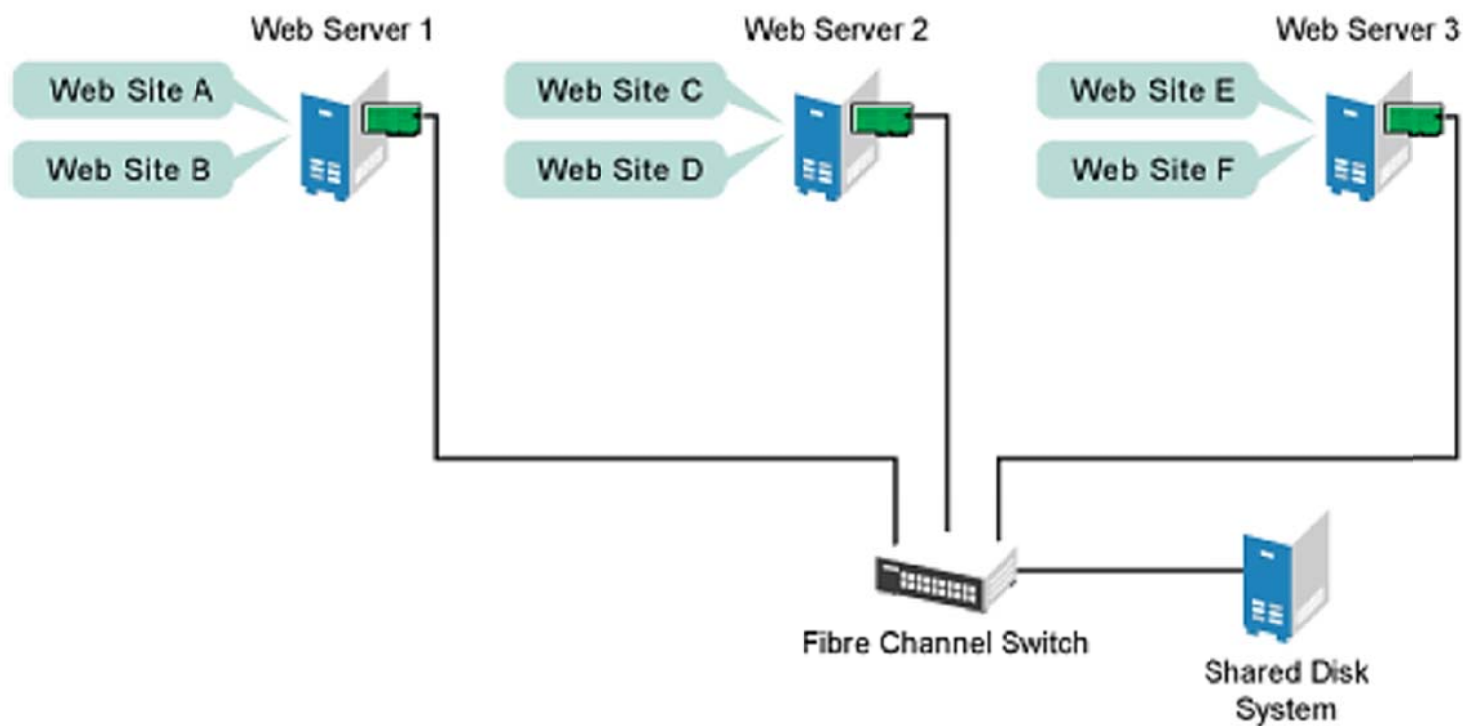
- Used in heavy-demand environments, such as very popular web sites.
- The purpose of a load balancing cluster is to redistribute a task to a server that has resources to handle the task.
- That seems a bit like high performance clustering, but the difference is that in high performance clusters, typically, all servers are working on the same task, where load balancing clusters take care of load distribution, to get an optimal efficiency in task-handling.
- A load balancing cluster consists of two entities: the load balancer and the server farm behind it.
- The load balancer receives requests from end users and redistributes them to one of the servers that is available in the server farm.
- The ClusterIP or Linux Virtual Server (LVS) projects implements load balancing clusters.
- See also recent trends on using eBPF for L4 LB, e.g. Katran from Facebook , Cloudflare's Unimog, or Cilium (<https://cilium.io/>)



High Availability Clusters:

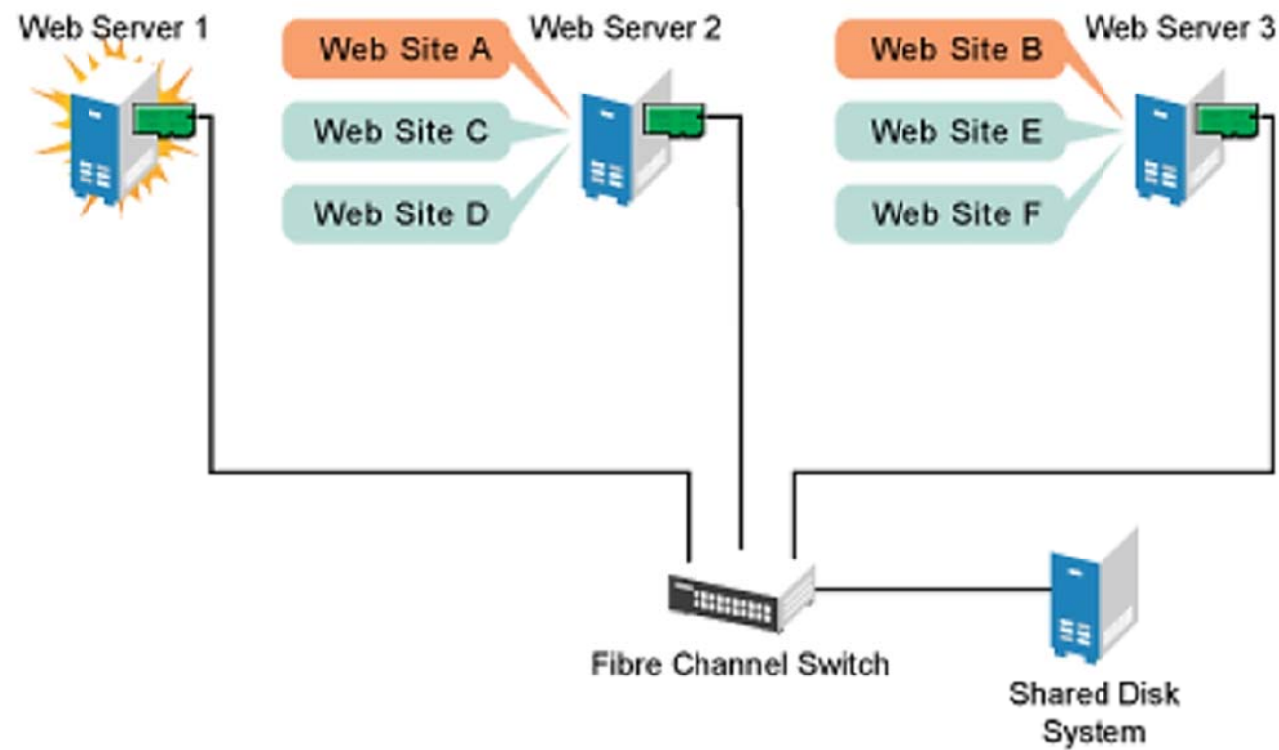
- The goal of a high availability cluster is to make sure that critical resources reach the maximum possible availability.
- This goal is accomplished by installing a software that monitors the availability of the cluster nodes / services / resources.
- If a server goes down, or the service stops, the HA cluster will make sure that the resource is restarted somewhere else in the cluster, so that it can be used again after a minimal interruption.
- The monitoring / management software often uses group membership and group communication mechanisms.
- For Linux clusters platform: Heartbeat, Red Hat Cluster Suite, Pacemaker.

HIGH AVAILABILITY CLUSTERS



HIGH AVAILABILITY CLUSTERS

(2)



LINUX HIGH AVAILABILITY CLUSTERS

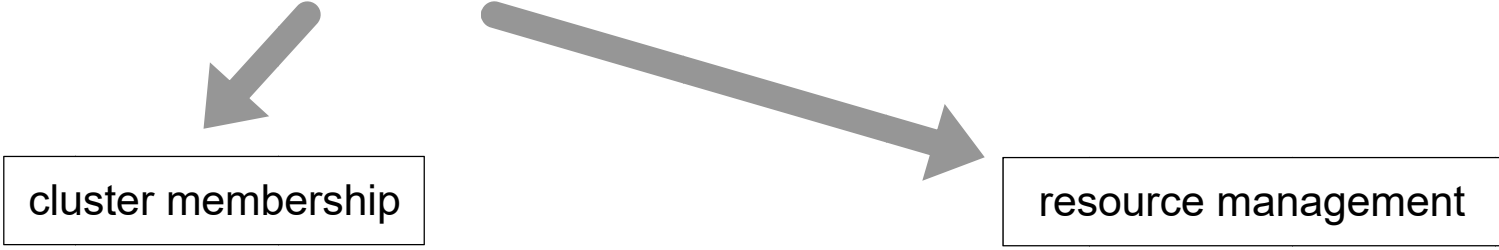
Heartbeat (1990)

- Heartbeat cluster basically could do two things: it monitored two nodes (and not more than two), and it was configured to start one or more services on those two nodes.
- If the node that was currently hosting the resources went down, it restarted the cluster resources on the remaining node.

Heartbeat 2.0 (2000)

<https://clusterlabs.org/>

- Added monitoring of resources and possibility to add more than two nodes.
- Finally Heartbeat 2.0 was split into two different projects:



cluster membership

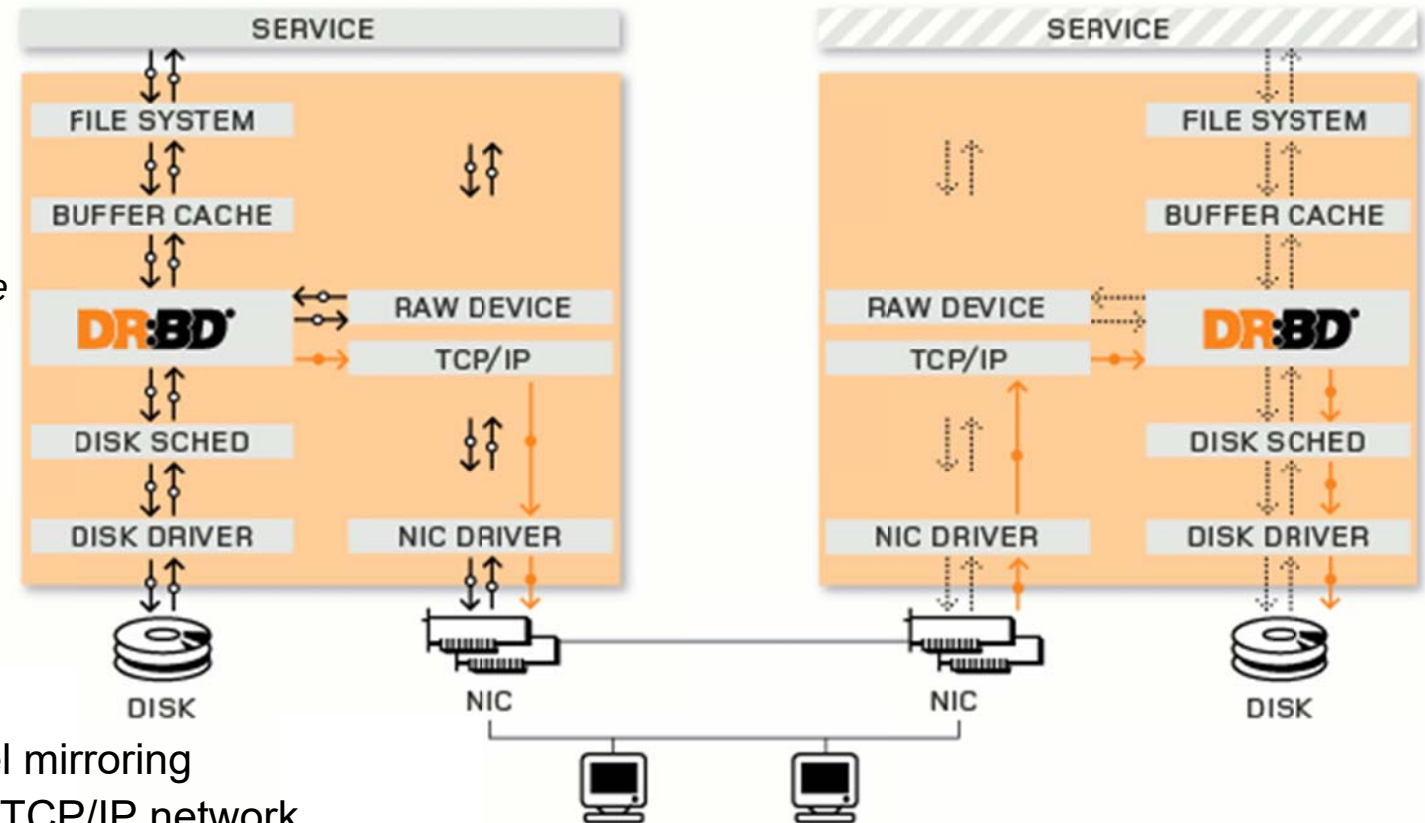
OpenAIS, CoroSycn, Red Hat cman

resource management

CRM, Pacemaker, Red Hat rgmanager



Distributed Replicated Block Device



<http://www.drbd.org>

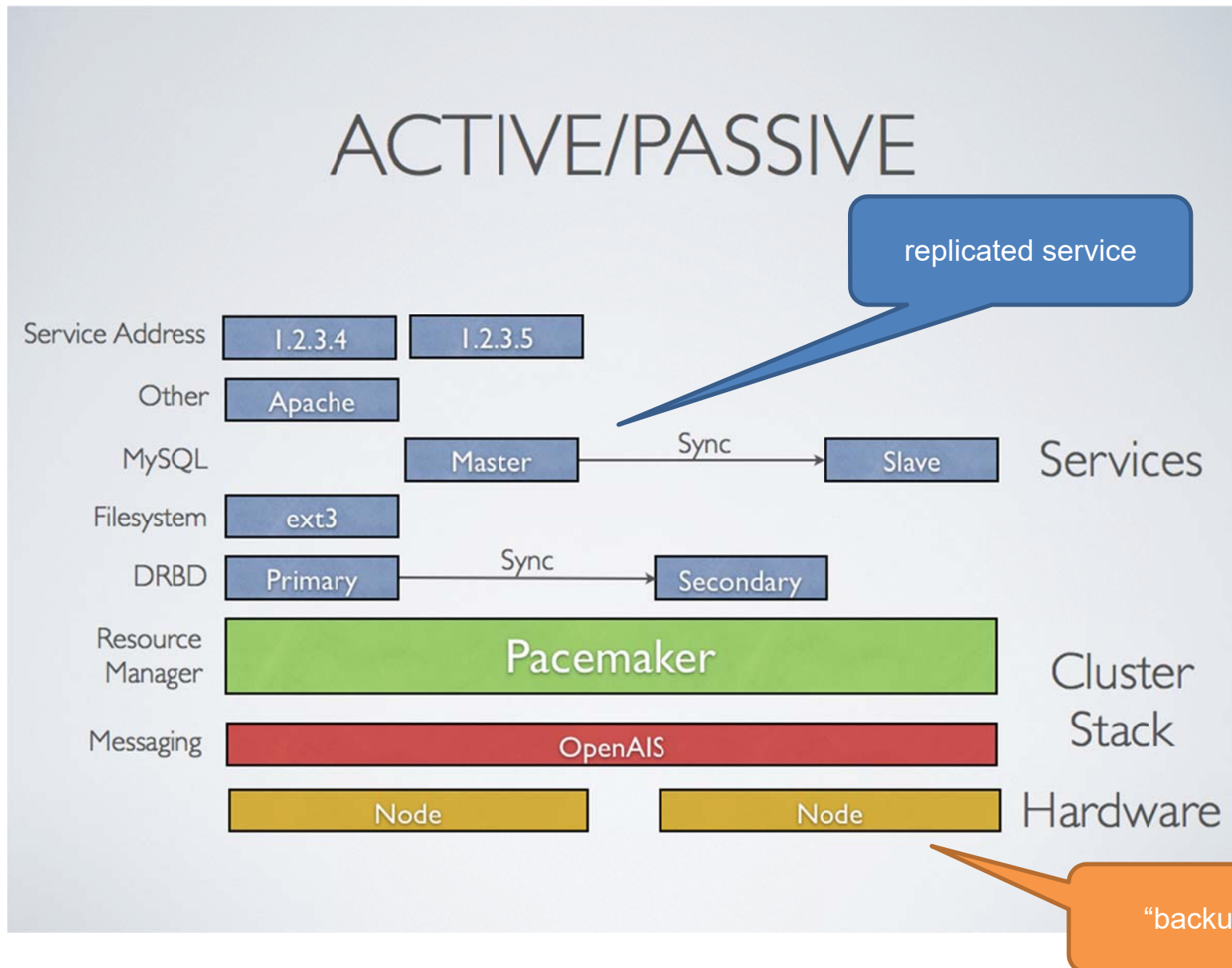
- DRBD refers to kernel-level mirroring a whole block device via a TCP/IP network.
- Can be understood as software network-based RAID1 primary-backup (or primary-primary).
- Allows to use “normal” filesystems like Ext4, XFS, Btrfs in the primary-backup configuration.

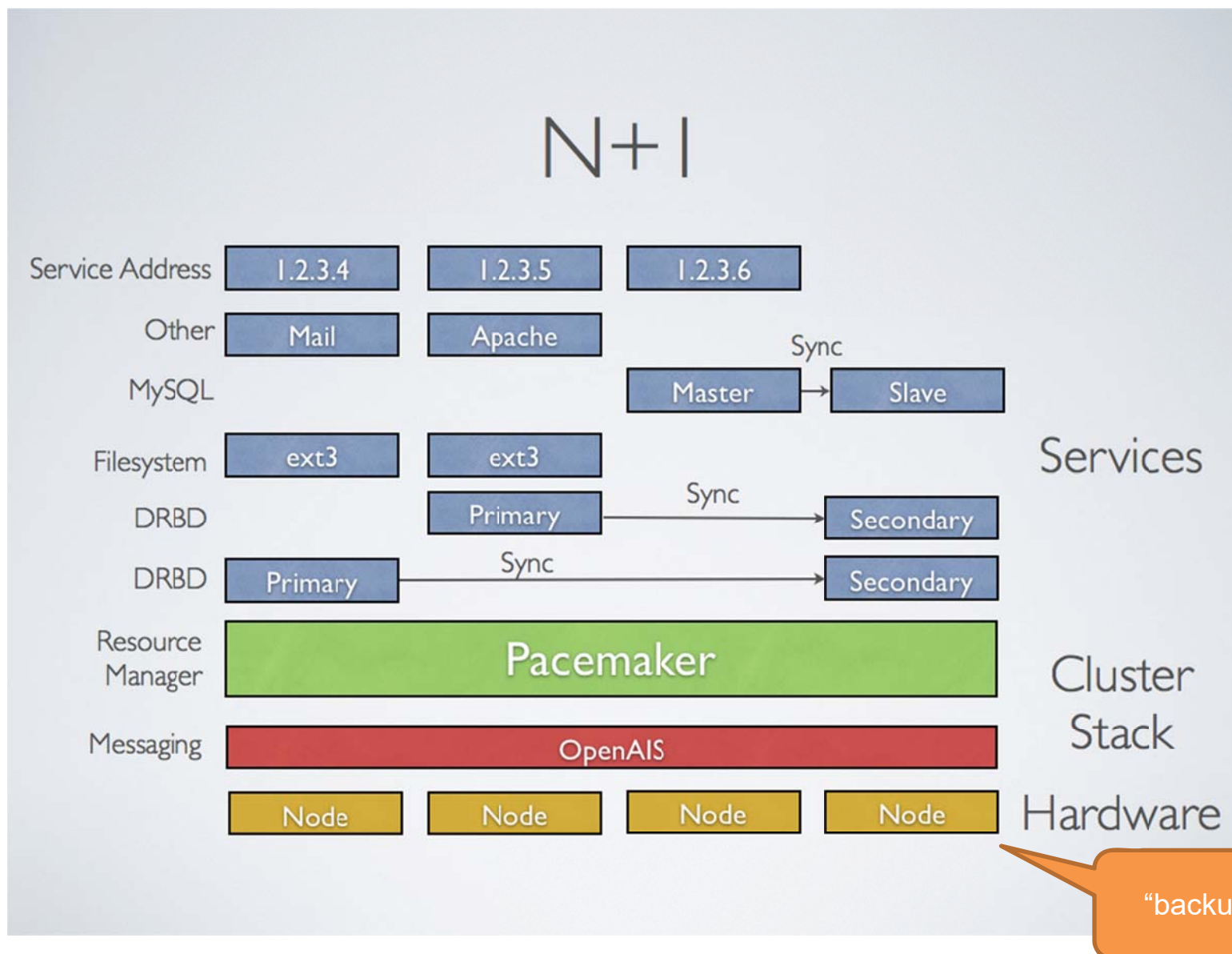
In the illustration above, the two orange boxes represent two nodes of an HA cluster. The black arrows illustrate the flow of data between cluster components. The orange arrows show the DRBD mirroring the storage data from the active node to the passive node of the HA cluster.

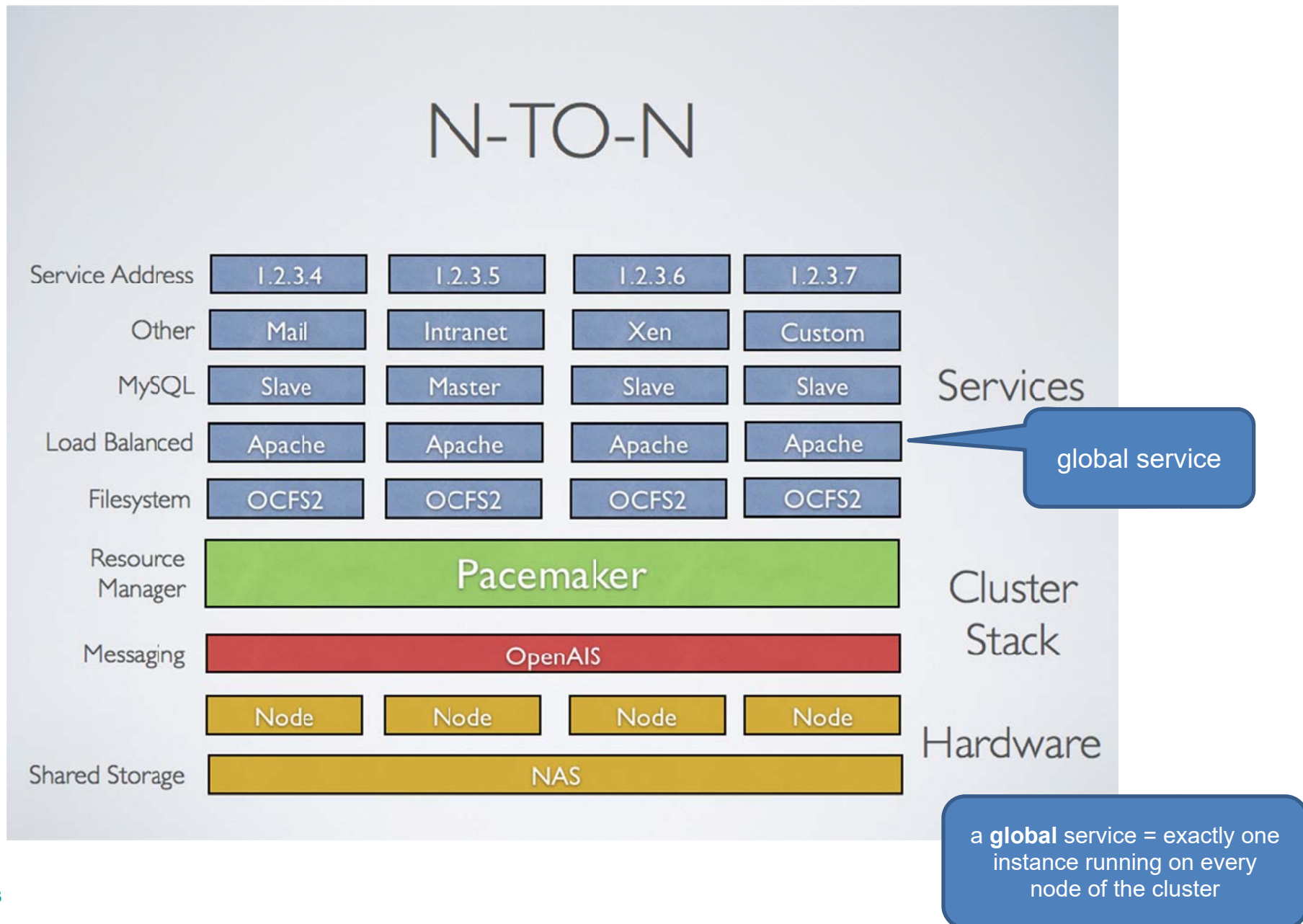


DRBD internals [3]:

- DRBD works by installing a kernel module on machines involved in the cluster.
- This kernel module picks up the I/O operations of writes just before they are scheduled for writing by the disk driver.
- Once the DRBD receives a write, it sends it via TCP/IP to the replica server, which itself sends the write to its local disk.
- At some stage during this process, the first node sends its write to its disk and reports that the write has been completed.
- There is a consistency versus performance trade-off, depending on exactly when the application is told that the write has succeeded.
- In *synchronous mode*, this will be done only after the write has been successfully made first onto the disk on the peer node, and then—on the calling node.





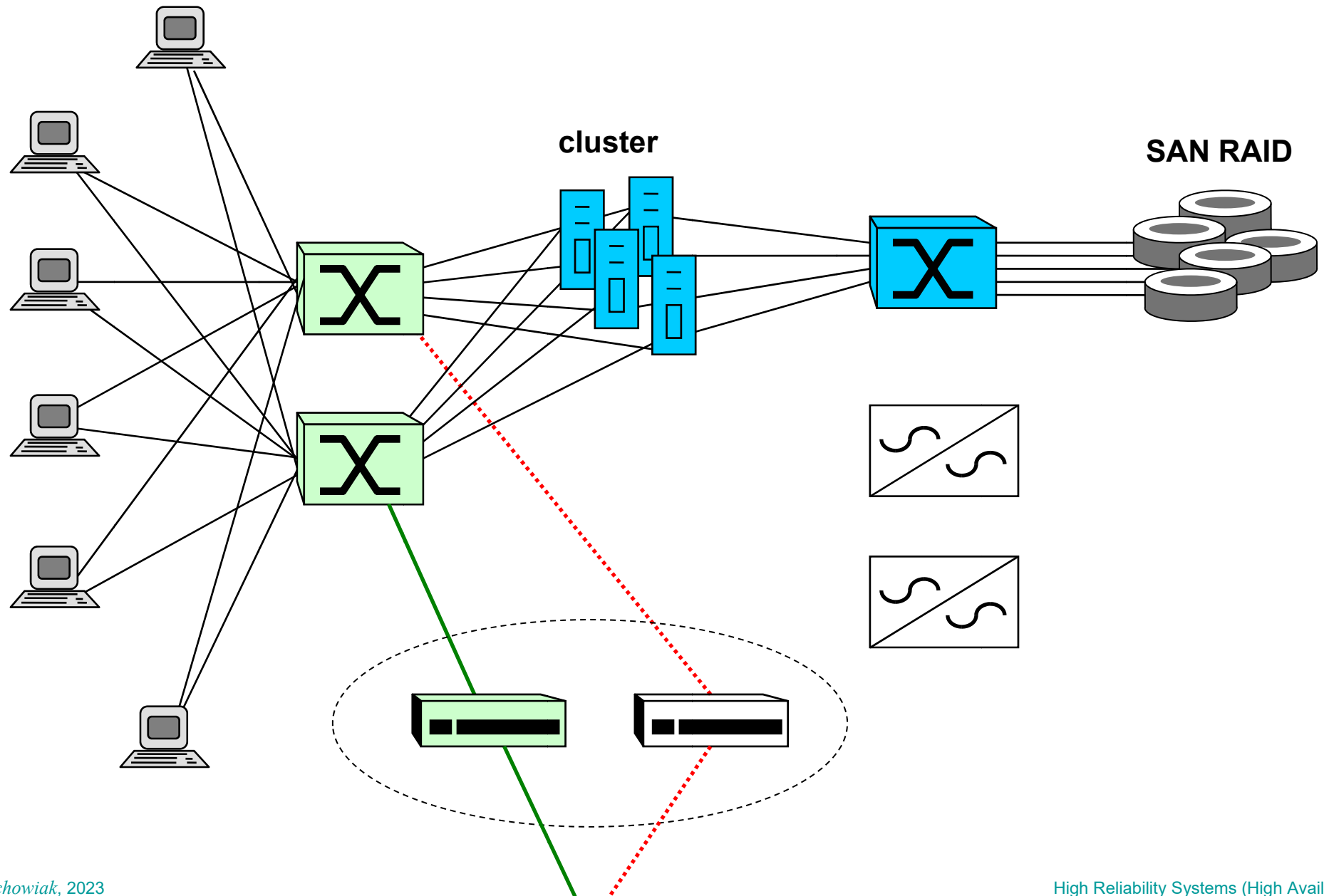




PODSUMOWANIE

REDUNDANCY AT A GLANCE

(1)



BIBLIOGRAPHY

- [1]  Peter S. Weygant, “Clusters for High Availability”, Prentice Hall, 2001
- [2]  Sander van Vugt, “Linux High Availability Clustering”, Apress, 2014
- [3]  Alex Davies, “High Availability MySQL Cookbook”, Packt Publishing, 2010
- [4]  Charles Bell, Mats Kindahl, Lars Thalmann, “MySQL High Availability”, O’Reilly, 2014
- [5]  Brendan Burns, "Designing Distributed Systems. Patterns and Paradigms for Scalable, Reliable Services", O'Reilly, 2018
- [6]  <https://clusterlabs.org/pacemaker/doc/>